

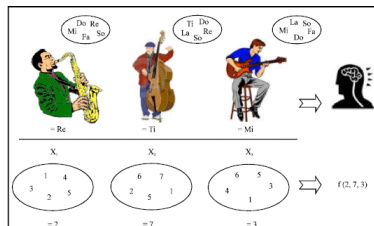


دانشگاه اصفهان

دانشکده مهندسی کامپیوتر - گروه هوش مصنوعی و رباتیک

گزارش تمرین سوم - بهینه‌سازی چندهدفه

سیستم توصیه‌گر HAS + MA



پدیدآورنده:

محمد امین کیانی

۴۰۴۳۶۴۴۰۰۸

دانشجوی کارشناسی ارشد، دانشکده‌ی کامپیوتر، دانشگاه اصفهان، اصفهان،
Aminkianiworkeng@gmail.com

استاد درس: دکتر کارشناس

نیمسال اول تحصیلی ۱۴۰۴-۰۵

فهرست مطالب

مستندات	۳
۰- مسئله و تحلیل کلی آن:	۳
۱- وظیفه اول: مطالعه و تحلیل الگوریتم‌ها	۳
الگوریتم جستجوی هارمونی (HSA)	۳
الگوریتم ممیتیک (MA)	۹
۲- وظیفه دوم: تعریف بهینه‌سازی چند هدفه	۱۷
نمادگذاری و داده‌های ورودی	۱۷
متغیر تصمیم و فضای جستجو	۱۸
توابع هدف	۱۹
قید حداقل پوشش ژانری	۲۰
نرمال‌سازی اهداف	۲۳
مدیریت چندهدفه (ترکیب/انتخاب راه‌حل‌ها)	۲۳
(Bias / Fairness / Novelty) - اهداف اضافی پیشنهادی	۲۴
۳- وظیفه سوم: پیاده‌سازی و تحلیل نتایج	۲۸
الگوریتم جستجوی هارمونی (HSA)	۳۱
الگوریتم ممیتیک (MA)	۳۵
۴- وظیفه چهارم: مقایسه گروهی و بحث نهایی	۴۶
الگوریتم جستجوی هارمونی (HSA)	۴۶
الگوریتم ممیتیک (MA)	۴۷
۵- مراجع	۵۰

مستندات

۰- مسئله و تحلیل کلی آن:

در اینجا به بررسی دو الگوریتم جستجوی هارمونی (Harmony Search Algorithm – HSA) و ممتیک (Memetic Algorithm – MA) می‌پردازیم. هر دو الگوریتم از نوع فراابتکاری هستند که برای حل مسائل بهینه‌سازی به کار می‌روند. ابتدا ماهیت هر الگوریتم را توضیح داده و سپس اجزا و عملگرهای اصلی آن‌ها را معرفی می‌کنیم و در ادامه نیز شبه‌کد نسخه استاندارد تک‌هدفی هر کدام ارائه شده است. در فازهای بعد به پیاده‌سازی و تحلیل نتایج الگوریتم‌ها پرداخته و در نهایت مقایسه‌ی گروهی برای پایان این روال صورت می‌گیرد.

۱- وظیفه اول: مطالعه و تحلیل الگوریتم‌ها

الگوریتم جستجوی هارمونی (HSA)

ماهیت الگوریتم: الگوریتم جستجوی هارمونی یک روش بهینه‌سازی تکاملی الهام‌گرفته از روند بداهه‌نوازی موسیقی است. این الگوریتم نخستین بار توسط Geem و همکاران در سال ۲۰۰۱ ارائه شد و به دلیل سادگی مفهومی، تعداد کم پارامترها و عدم نیاز به مشتق یا گرادیان، به یکی از روش‌های پرکاربرد بهینه‌سازی تبدیل شده است. ایده کلی HSA تشبیه فرایند پیدا کردن ترکیب‌های موسیقایی هماهنگ (هارمونیک) به فرایند جستجوی راه‌حل‌های بهینه در مسائل ریاضی است. به جای تأکید بر استعاره موسیقی، در اینجا بر مکانیزم عملی الگوریتم تمرکز می‌کنیم: HSA با نگهداری مجموعه‌ای از راه‌حل‌ها (که **حافظه هارمونی** نامیده می‌شود) تلاش می‌کند ترکیبی بهینه از تصمیم‌ها را با استفاده از انتخاب از حافظه، تنظیم (جهش) جزئی مقادیر و **کاوش تصادفی** بیابد. (HSA از اطلاعات تمام جمعیت برای تولید فرزند جدید بهره می‌برد که نوعی "باز ترکیب جهانی" (Global Recombination) محسوب می‌شود).

اجزا و عملگرهای اصلی در HSA : الگوریتم جستجوی هارمونی دارای مولفه‌ها و پارامترهای کلیدی زیر است که رفتار جستجو را تعیین می‌کنند:

- **حافظه هارمونی (Harmony Memory – HM) :** مجموعه‌ای از راه‌حل‌های فعلی (بردارهای تصمیم) است که الگوریتم آن‌ها را حفظ می‌کند. اندازه این حافظه با پارامتری به نام HMS (Harmony Memory Size) مشخص می‌شود. هر راه‌حل را می‌توان به یک قطعه موسیقی (یک بردار هارمونیک) تشبیه کرد. الگوریتم با ترکیب اطلاعات راه‌حل‌های موجود در HM راه‌حل‌های جدید می‌سازد.
- **نرخ ملاحظه حافظه (Harmony Memory Consideration Rate – HMCR) :** احتمالی بین ۰ و ۱ که تعیین می‌کند هر متغیر تصمیم در راه‌حل جدید چگونه انتخاب شود. با احتمال HMCR ، مقدار یک متغیر جدید از بین مقادیر موجود در حافظه هارمونی انتخاب می‌شود (استفاده از دانش راه‌حل‌های قبلی) و یک مقدار کاملاً تصادفی برای آن متغیر از فضای جستجو انتخاب خواهد شد. این مکانیزم نوعی انتخاب مبتنی بر حافظه است که تضمین می‌کند الگوریتم از تجربیات گذشته استفاده کند.
- **نرخ تنظیم گام (Pitch Adjustment Rate – PAR) و پهنای باند (Bandwidth – bw) :** پس از انتخاب یک مقدار از حافظه برای متغیر (طبق HMCR) ، با احتمال PAR آن مقدار دستخوش یک تغییر جزئی می‌شود. این تغییر به اندازه کوچکی (مثلاً اضافه/کم کردن یک مقدار دلتا) صورت می‌گیرد که میزان حداکثر تغییر توسط پارامتر **bw** تعیین می‌شود. به این عملگر اصطلاحاً **تنظیم گام** می‌گویند که مشابه عمل جهش (Mutation) در الگوریتم‌های تکاملی است. PAR معمولاً یک مقدار کوچک (مثلاً ۰.۱ یا ۰.۲) در نظر گرفته می‌شود تا تنظیمات جزئی موجب **جستجوی محلی** پیرامون راه‌حل‌های موجود شود. ترکیب PAR و bw امکان کاوش دقیق‌تر در حوالی راه‌حل‌های فعلی را فراهم می‌کند.
- **انتخاب تصادفی:** زمانی که انتخاب از حافظه انجام نمی‌شود، مقدار یک متغیر به صورت کاملاً تصادفی از دامنه ممکن متغیر برگزیده می‌شود. این مؤلفه نقشی شبیه **کاوش جهانی**

(Exploration) را ایفا می کند و تنوع جمعیت را بالا نگه می دارد تا الگوریتم در بهینه های محلی گرفتار نشود. در واقع تعادل بین استفاده از حافظه (استفاده از دانش کسب شده، استثمار یا Exploitation) و انتخاب تصادفی (کاوش موارد جدید، Exploration) از طریق تنظیم HMCR برقرار می شود.

- **تابع هدف (Objective Function) و ارزیابی:** هر راه حل (هارمونی) از طریق یک تابع شایستگی یا هدف ارزیابی می شود. الگوریتم تک هدفه HSA به دنبال بیشینه سازی یا کمینه سازی این تابع هدف برای پیدا کردن بهترین راه حل است. مقدار این تابع برای هر هارمونی تعیین کننده کیفیت آن راه حل است.

- **به روز رسانی حافظه:** حافظه هارمونی همواره فقط تعداد محدودی راه حل (HMS عدد) را نگهداری می کند. پس از تولید هر راه حل جدید و محاسبه شایستگی آن، اگر این راه حل از بدترین راه حل حاضر در HM بهتر باشد، جایگزین آن می شود. این مکانیزم مشابه بقا و انتخاب طبیعی است که تضمین می کند کیفیت کلی حافظه در طول زمان بهبود یابد. به عبارت دیگر، HM همیشه تمایل دارد بهترین راه حل های یافت شده تاکنون را حفظ کند.

- **شرط توقف:** الگوریتم جستجوی هارمونی تا زمانی که شرط توقف برقرار نشده باشد، به تولید راه حل جدید و به روز رسانی حافظه ادامه می دهد. شروط توقف رایج شامل رسیدن به تعداد معینی تکرار/ارزیابی، یافتن راه حلی با کیفیت مطلوب یا عدم بهبود قابل توجه طی چندین تکرار اخیر است.

شبه کد نسخه تک هدفه استاندارد HSA: روند کلی الگوریتم جستجوی هارمونی در حالت بهینه سازی تک هدفه به صورت زیر خلاصه می شود:

الگوریتم جستجوی هارمونی (تک هدفه)

۱. تنظیم پارامترهای اولیه:

- تعیین اندازه حافظه هارمونی (HMS)
- تعیین HMCR, PAR, bw و سایر پارامترهای مسئله (مانند دامنه متغیرها)
- تعریف تابع هدف برای ارزیابی کیفیت راحل‌ها

۲. مقداردهی اولیه حافظه هارمونی (HM):

- تولید HMS راحل اولیه به صورت تصادفی
- ارزیابی هر راحل و ذخیره در حافظه هارمونی

۳. تکرار فرایند بدست آوردن هارمونی جدید تا زمان ارضای شرط توقف:

الف. **تولید هارمونی جدید**:

- برای هر متغیر تصمیم x_i^* در بردار راحل:
- تولید یک عدد تصادفی یکتا در بازه $[0,1]$.
- اگر عدد تصادفی $> \text{HMCR}$:

مقدار x_i^* را از بین مقادیر قبلی موجود در ستون x_i^* ام حافظه هارمونی انتخاب کن (انتخاب از HM).

در غیر این صورت:

- مقدار x_i^* را به صورت تصادفی از دامنه معتبر متغیر انتخاب کن.
- اگر مقدار x_i^* از حافظه انتخاب شده بود:
- یک عدد تصادفی جدید در $[0,1]$ تولید کن.
- اگر این عدد $> \text{PAR}$:

مقدار x_i^* را با یک تغییر جزئی تنظیم کن (افزایش یا کاهش کوچک حداکثر به اندازه bw).

ب. **ارزیابی هارمونی جدید**:

مقدار تابع هدف را برای بردار تصمیم جدید محاسبه کن.

پ. **بهروزرسانی حافظه هارمونی**:

- اگر شایستگی (کیفیت) هارمونی جدید بهتر از بدترین هارمونی فعلی در HM باشد:
- هارمونی جدید را جایگزین بدترین هارمونی در حافظه کن (حافظه را بهروز کن).

ت. **ادامه حلقه**:

مراحل تولید تا بهروزرسانی را تا زمانی که شرط پایان (مثلاً تعداد تکرار معین یا رسیدن به بهینگی مطلوب) برآورده شود تکرار کن.

۴. خاتمه الگوریتم:

- به عنوان خروجی، بهترین راحل موجود در حافظه هارمونی (یا مجموعه‌ای از بهترین‌ها) ارائه می‌شود.

1) Harmony Search Algorithm (HSA) — Single-Objective Standard Pseudocode

Assumptions / Inputs

- Objective function: $f(x)$ (either **minimize** or **maximize**)
- Decision vector: $x = (x_1, x_2, \dots, x_N)$ with domains $D_1, D_2, \dots, D_N$
- Parameters:
 - **HMS** = Harmony Memory Size (population size)
 - **HMCR** $\in [0,1]$ = Harmony Memory Consideration Rate
 - **PAR** $\in [0,1]$ = Pitch Adjustment Rate
 - **bw** > 0 = bandwidth (step size for pitch adjustment; used when meaningful)
 - **MaxIters** = maximum improvisation iterations
- Comparator:
 - **Better(a,b)** returns true if solution **a** is better than **b** (depends on min/max)

Helper Functions

- **RandomSample(D_i)** $\rightarrow$ sample a valid value from domain D_i
- **PitchAdjust(value, D_i , bw)** $\rightarrow$ returns a *neighbor* value of **value** within D_i
- **ArgWorst(HM)** $\rightarrow$ index of worst harmony in memory (by objective)
- **ArgBest(HM)** $\rightarrow$ index of best harmony in memory (by objective)

Pitch Adjustment (generic)

You must implement this depending on variable type:

- **Continuous:** $\text{value}' = \text{clamp}(\text{value} + \text{Uniform}(-bw, +bw), D_i)$
- **Integer:** $\text{value}' = \text{clamp}(\text{value} + \text{RandomChoice}(\{-1,+1\}) * \text{step}, D_i)$ (e.g., **step=1** or derived from **bw**)
- **Categorical / permutation:** apply a small neighborhood move (swap, insert, replace, etc.)

Algorithm **HSA_SingleObjective**($f, \{D_i\}_{i=1..N}, \text{HMS}, \text{HMCR}, \text{PAR}, \text{bw}, \text{MaxIters}$):

```
# -----
# 1) Initialize Harmony Memory
# -----
HM = empty list
for j = 1 to HMS:
    x = empty vector of length N
    for i = 1 to N:
        x[i] = RandomSample(Di)
    x.f = f(x)
    HM.add(x)
```

```

# (Optional) keep HM sorted by quality to speed up best/worst access

best = HM[ArgBest(HM)]

# -----
# 2) Main improvisation loop
# -----
for iter = 1 to MaxIters:
    # 2.a) Improvise a new harmony x_new
    x_new = empty vector length N

    for i = 1 to N:
        r = Uniform(0, 1)

        if r < HMCR:
            # Memory consideration: pick a value from existing HM for dimension i
            # Option A: pick a random harmony and copy its i-th variable
            h = RandomInteger(1, HMS)
            x_new[i] = HM[h][i]

            # Pitch adjustment with probability PAR
            r2 = Uniform(0, 1)
            if r2 < PAR:
                x_new[i] = PitchAdjust(x_new[i], Di, bw)
        else:
            # Random consideration: sample from domain
            x_new[i] = RandomSample(Di)

    # 2.b) Evaluate
    x_new.f = f(x_new)

    # 2.c) Update Harmony Memory (replace worst if new is better)
    w = ArgWorst(HM)
    if Better(x_new, HM[w]):
        HM[w] = x_new

    # 2.d) Track best
    b = ArgBest(HM)
    if Better(HM[b], best):
        best = HM[b]

# -----
# 3) Return best solution found
# -----
return best

```


در شبه کد فوق، مشاهده می‌شود که الگوریتم HSA چگونه با ترکیب سه گزینه (انتخاب از حافظه، تنظیم مقداری کوچک، یا انتخاب کاملاً تصادفی) یک راه‌حل جدید می‌سازد. این روش سبب می‌شود که هم تنوع راه‌حل‌ها حفظ شود (از طریق انتخاب‌های تصادفی) و هم از تجربیات قبلی استفاده بهینه شود (از طریق حافظه هارمونی). در نسخه تک‌هدفه، معیار "بهترین" بودن هارمونی بر اساس مقدار عددی تابع هدف است؛ یعنی الگوریتم یک راه‌حل بهینه‌ی نهایی را بازمی‌گرداند. در مسائل چندهدفه (که در وظایف بعدی مد نظر است)، به جای بهترین تک‌راه‌حل، مجموعه‌ای از راه‌حل‌های غیراست **dominated** (جبهه‌ی پارتو) مدنظر خواهد بود که در ادامه به آن پرداخته می‌شود.

الگوریتم ممتیک (MA)

ماهیت الگوریتم: الگوریتم ممتیک یک روش بهینه‌سازی تکاملی ترکیبی است که با افزودن جستجوی محلی به رویه‌های یک الگوریتم ژنتیک (GA) استاندارد، کارایی جستجو را بهبود می‌دهد. نام "ممتیک" از مفهوم میم (واحد انتقال فرهنگی یا ایده) گرفته شده که توسط ریچارد داوکینز معرفی شد و در اینجا به این معناست که هر راه‌حل (هر فرد در جمعیت) می‌تواند مستقل از دیگران تکامل محلی پیدا کند. به عبارت دیگر، MA را می‌توان الگوریتم ژنتیک به همراه بهبود موضعی در نظر گرفت؛ از این رو به آن **الگوریتم تکاملی هیبریدی** نیز می‌گویند. این الگوریتم تلاش می‌کند مزایای جستجوی سراسری تکاملی (که تنوع زیادی ایجاد می‌کند) را با مزایای جستجوی محلی (که به صورت موضعی راه‌حل را بهبود می‌بخشد) ترکیب کند تا به کیفیت بهتری در حل مسئله برسد. در ادامه اجزای اصلی MA را بررسی می‌کنیم.

اجزا و عملگرهای اصلی در MA : الگوریتم ممتیک شامل تمامی مولفه‌های یک الگوریتم ژنتیک استاندارد به علاوه یک مولفه‌ی جستجوی محلی است:

- **نمایش راه‌حل و جمعیت اولیه:** مانند سایر الگوریتم‌های تکاملی، ابتدا باید نحوه نمایش (کدگذاری) هر راه‌حل مشخص شود (مثلاً یک بردار اعداد حقیقی، یک رشته باینری، یک دنباله ترتیب آیت‌ها و غیره). سپس یک **جمعیت اولیه** از راه‌حل‌ها (معمولاً به صورت

تصادفی) ایجاد می‌شود. اندازه جمعیت یک پارامتر مهم است. هر عضو جمعیت را می‌توان مشابه یک "فرد" یا یک کاندید پاسخ در نظر گرفت.

- **تابع هدف و ارزیابی:** یک تابع شایستگی (هدف) برای سنجش کیفیت هر فرد تعریف می‌شود. در حالت تک‌هدفه، این تابع ممکن است امتیازی برای مطلوبیت راه‌حل باشد که الگوریتم سعی در بیشینه کردن (یا کمینه کردن) آن دارد. تمامی افراد جمعیت اولیه پس از ایجاد، توسط تابع هدف ارزیابی و مقدار شایستگی‌شان مشخص می‌شود.

- **انتخاب والدین (Selection):** در هر نسل، تعدادی از افراد به عنوان **والدین** برای تولید فرزندان جدید انتخاب می‌شوند. روش‌های مختلفی برای انتخاب وجود دارد (همانند الگوریتم ژنتیک)، از جمله **چرخ رولت** (متناسب با شایستگی)، **تورنمنت** (بین چند انتخاب تصادفی، بهترین برگزیده می‌شود) و **یا انتخاب رتبه‌بندی شده**. هدف مرحله انتخاب این است که به راه‌حل‌های بهتر شانس بیشتری برای تولید نسل بعدی بدهیم، در عین حال تنوع ژنتیکی نیز حفظ شود.

- **ترکیب (تقاطع یا Crossover):** عملگر ترکیب روی والدین انتخاب‌شده اعمال می‌شود تا **فرزندان** جدید ایجاد شوند. این عملگر بخش‌هایی از اطلاعات دو (یا چند) والد را با هم ترکیب می‌کند. برای مثال، در نمایش برداری ممکن است یک نقطه‌ی برش تعیین شود و بخش ابتدایی بردار از والد اول و بخش انتهایی از والد دوم گرفته شود (یک نقطه‌ای یا دونقطه‌ای)، یا در نمایش‌های دیگر، روش‌های ترکیب خاصی تعریف می‌گردد. نتیجه، تولید راه‌حل‌های جدیدی است که ترکیبی از خصوصیات والدین هستند. نرخ ترکیب معمولاً یک پارامتر بین ۰ و ۱ است (مثلاً ۰.۸) که تعیین می‌کند با چه احتمالی یک جفت والدین واقعاً ترکیب شوند.

- **جهش (Mutation):** پس از ایجاد فرزندان توسط ترکیب، یک عملگر جهش روی فرزندان اعمال می‌شود تا تغییرات تصادفی کوچکی در آن‌ها ایجاد کند. جهش تضمین می‌کند که تنوع در جمعیت حفظ شده و الگوریتم در یک مسیر تکاملی محدود گیر نیفتد. میزان و نوع

جهش وابسته به کدگذاری راه حل است (مثلاً تغییر تصادفی یک بیت در رشته باینری، یا تغییر اندک یک مقدار در بردار حقیقی، یا جابجایی دو عنصر در یک دنباله). نرخ جهش نیز یک پارامتر الگوریتم (معمولاً کوچک، مثلاً ۰.۰۱ یا ۰.۰۵) است که احتمال اعمال جهش روی هر فرد یا هر ژن را تعیین می کند.

- **جستجوی محلی (Local Search):** این مولفه وجه تمایز اصلی الگوریتم ممتیک از یک GA ساده است. پس از ایجاد هر فرزند (یا پس از انتخاب برخی از فرزندان)، یک رویه ی بهبود موضعی روی آن فرد اجرا می شود. جستجوی محلی می تواند هر الگوریتم بهینه سازی تک نقطه ای باشد که از پاسخ فعلی شروع شده و سعی می کند یک بهبود کوچک در آن ایجاد کند. برای مثال، اگر مسئله پیوسته باشد می توان یک گام گرادیانی کوچک (در صورت محاسبه پذیر بودن) یا تغییرات تصادفی محدود در همسایگی انجام داد؛ یا اگر مسئله گسسته باشد می توان از عملگرهای تبادل یا حرکتهای کوچک در فضای همسایگی بهره برد. این مرحله تضمین می کند که فرزندان تولیدشده فوراً **بهبود کیفی محلی** پیدا کنند و تا حد امکان در ناحیه پیرامون خود بهینه تر شوند. معمولاً اعمال جستجوی محلی برای همه فرزندان ممکن است زمان بر باشد، بنابراین گاهی فقط روی بهترین فرزندان یا به صورت دوره ای انجام می شود. انتخاب نوع و شدت جستجوی محلی یکی از مباحث طراحی در MA است که باید با فرایند تکاملی کلی متعادل شود (اگر جستجوی محلی بیش از حد قوی باشد، ممکن است تنوع را کاهش داده و الگوریتم در بهینه محلی گیر بیفتد).

- **جایگزینی (Replacement) و به روزرسانی جمعیت:** پس از ایجاد و بهبود فرزندان، کل جمعیت باید برای نسل بعد به روزرسانی شود. در ساده ترین حالت (الگوریتم ژنتیک سنتی) کل والدین با فرزندان جدید **جایگزین** می شوند (نسل جدید کاملاً جانشین نسل قبل می گردد). اما در بسیاری از موارد، به ویژه در MA، از استراتژی های پیشرفته تر استفاده می شود تا اطلاعات مفید از نسل قبل حفظ شود؛ برای مثال ممکن است **نخبه گرایی (Elitism)** به کار رود که طی آن یکی چند تا از بهترین راه حل های نسل قبل مستقیماً به نسل بعد منتقل می شوند تا از بین نروند. یا ممکن است تعداد افراد جمعیت ثابت نگه داشته

شود و ترکیبی از بهترین والدین و فرزندان، جمعیت جدید را تشکیل دهند. هدف این مرحله آن است که جمعیت نسل بعد همچنان دارای اندازه ثابت باشد و آماده تکرار فرآیند تکاملی در گام بعدی شود.

- **شرط توقف:** مشابه دیگر الگوریتم‌های تکاملی، حلقه تولید نسل‌ها در MA نیز تا زمانی ادامه می‌یابد که شرط(های) پایان ارضا شوند. شرط توقف می‌تواند رسیدن به تعداد مشخصی نسل/ارزیابی، رسیدن به سطح مطلوب تابع هدف، یا عدم بهبود محسوس طی چند نسل اخیر باشد. پس از توقف، به‌طور معمول بهترین فرد یافت‌شده یا کل جمعیت نهایی (برای تحلیل) خروجی داده می‌شود.

شبه‌کد نسخه تک‌هدفه استاندارد MA: الگوریتم ممیتیک در حالت تک‌هدفه (مثلاً بیشینه‌سازی یک تابع معین) را می‌توان به صورت زیر خلاصه کرد:

الگوریتم ممیتیک (تک‌هدفه)

۱. مقداردهی اولیه:
 - تعریف نحوه نمایش راحل (کدگذاری کروموزوم‌ها)
 - تنظیم پارامترها (اندازه جمعیت N ، نرخ ترکیب pc ، نرخ جهش pm و ...)
 - تولید جمعیت اولیه شامل N راحل (تصادفی یا با استفاده از دانش مسئله)
 - ارزیابی اولیه جمعیت (محاسبه تابع هدف برای هر فرد)
۲. حلقه تکاملی (تا زمانی که شرط توقف برقرار نیاشد):
 - الف. ****انتخاب والدین****
 - با توجه به مقادیر شایستگی، تعدادی والد از جمعیت جاری انتخاب کن (مثلاً به روش تورنمنت یا چرخ رولت).
 - ب. ****تولید فرزندان یا ترکیب****
 - والدین انتخاب‌شده را جفت کن و با احتمال pc عملگر ترکیب را روی هر جفت اعمال کن تا کروموزوم‌های فرزند تولید شوند.
 - پ. ****اعمال جهش****
 - روی هر کروموزوم فرزند با احتمال pm جهش (تغییر تصادفی کوچک) اعمال کن.
 - ت. ****جستجوی محلی روی فرزندان****
 - برای هر فرزند (یا برخی از فرزندان منتخب)، رویه بهبود محلی را اعمال کن تا کروموزوم اندکی بهینه‌تر شود.
 - ارزیابی مجدد فرزندان پس از جستجوی محلی (محاسبه تابع هدف به‌روز شده).
 - ث. ****تشکیل نسل جدید (جایگزینی)****
 - ترکیب والدین و فرزندان یا فقط استفاده از فرزندان برای به‌روزرسانی جمعیت:
 - * (حالت ساده) نسل جدید = تعداد N فرزند تازه ایجادشده (نسل قبلی کاملاً جایگزین می‌شود).
 - * (حالت نخیمگرا) چند فرد برتر از نسل قبل + بقیه از بهترین فرزندان = نسل جدید.
 - در هر صورت، اندازه جمعیت را به N فرد در نسل جدید تنظیم کن.
 - ج. ****به‌روزرسانی بهترین راحل پیدا شده (اختیاری)****
 - اگر لازم است، بهترین فرد فعلی را در جایی ذخیره کن (برای گزارش یا استفاده آتی).

۳. پایان الگوریتم:

- خروجی: بهترین راحل یا مجموعه‌ای از بهترین راحل‌های به‌دست‌آمده طی تکامل.

2) Memetic Algorithm (MA) — Single-Objective Standard Pseudocode

A Memetic Algorithm = Evolutionary Algorithm (e.g., GA) + Local Search (improvement) on individuals (often offspring).

Assumptions / Inputs

- Objective function: $f(x)$ (minimize or maximize)
- Representation: defines what an individual x looks like (binary, real vector, permutation, ranked list, ...)
- Parameters:
 - `PopSize` = population size N
 - $p_c \in [0,1]$ = crossover probability
 - $p_m \in [0,1]$ = mutation probability (per individual or per gene, depending on your design)
 - `MaxGens` = number of generations
 - `EliteSize` = number of elites carried over (elitism)
 - $p_{LS} \in [0,1]$ = probability of applying local search to an offspring (common design)
 - `LS_Budget` = maximum local search steps (or evaluations) per LS call
- Operators:
 - `InitializeIndividual()` → creates a random feasible solution
 - `SelectParent(P)` → e.g., tournament selection
 - `Crossover(p1, p2)` → returns children (`c1`, `c2`) (type depends on representation)
 - `Mutate(x, pm)` → mutated individual
 - `LocalSearch(x, f, LS_Budget)` → locally improved individual
- Comparator:
 - `Better(a,b)` like above

A Generic Local Search Template :

```
Procedure LocalSearch_HillClimb(x, f, LS_Budget):
    x_best = x
    f_best = f(x_best)

    for step = 1 to LS_Budget:
        x_n = GenerateNeighbor(x_best)      # small move in neighborhood
        f_n = f(x_n)

        if Better( (x_n, f_n), (x_best, f_best) ):
            x_best = x_n
            f_best = f_n
```

```
return x_best with fitness f_best
```

MA Pseudocode (Generational + Elitism + LS on offspring):

Algorithm MA_SingleObjective(f, PopSize, pc, pm, MaxGens, EliteSize, pLS, LS_Budget):

```
# -----
# 1) Initialize population
# -----
P = empty list
for i = 1 to PopSize:
    x = InitializeIndividual()
    x.f = f(x)
    P.add(x)

best = BestIndividual(P) # by Better()

# -----
# 2) Evolutionary loop
# -----
for gen = 1 to MaxGens:

    # 2.a) Create next generation
    P_next = empty list

    # --- Elitism: copy top EliteSize individuals
    elites = TopK(P, EliteSize) # sorted by quality
    P_next.add_all(elites)

    # --- Generate offspring until population is full
    while |P_next| < PopSize:

        # Selection
        p1 = SelectParent(P)
        p2 = SelectParent(P)

        # Crossover
        r = Uniform(0,1)
        if r < pc:
            (c1, c2) = Crossover(p1, p2)
        else:
            c1 = Clone(p1)
```

```

        c2 = Clone(p2)

    # Mutation
    c1 = Mutate(c1, pm)
    c2 = Mutate(c2, pm)

    # Local Search (Memetic part)
    r1 = Uniform(0,1)
    if r1 < pLS:
        c1 = LocalSearch(c1, f, LS_Budget)
    else:
        c1.f = f(c1)

    r2 = Uniform(0,1)
    if r2 < pLS:
        c2 = LocalSearch(c2, f, LS_Budget)
    else:
        c2.f = f(c2)

    # Add offspring (respect PopSize)
    if |P_next| < PopSize:
        P_next.add(c1)
    if |P_next| < PopSize:
        P_next.add(c2)

    # 2.b) Replace population
    P = P_next

    # 2.c) Update global best
    current_best = BestIndividual(P)
    if Better(current_best, best):
        best = current_best

    # (Optional) early stopping if no improvement for T generations, etc.

    # -----
    # 3) Return best found
    # -----
    return best

```

در این شبه‌کد، مشاهده می‌شود که MA همان ساختار کلی الگوریتم ژنتیک را دنبال می‌کند (مراحل انتخاب، ترکیب، جهش، جایگزینی)، با این تفاوت مهم که مرحله‌ی ت. جستجوی محلی روی فرزندان اضافه شده است. این مرحله اجازه می‌دهد هر راه‌حل به صورت فردی بهبود یابد؛ به

عبارتی هر "میم" (راه حل) می تواند مستقل یاد بگیرد و بهتر شود. نتیجه این است که الگوریتم معمولاً با سرعت همگرایی بیشتری به راه حل های خوب می رسد، زیرا کاوش سراسری GA برای نقاط امیدوارکننده از طریق جستجوی محلی تثبیت و تقویت می شود. البته باید توجه داشت که هزینه محاسباتی MA معمولاً بیشتر از GA معمولی است (به دلیل صرف زمان برای جستجوی محلی در هر نسل)، بنابراین یکی از ملاحظات مهم، تعادل بین شدت جستجوی محلی و اندازه/تعداد نسل های تکاملی است.

به صورت خلاصه:

- **در HSA:** جستجو از طریق ترکیب ویژگی های موجود در حافظه انجام می شود. برای مسئله Top-K این یعنی الگوریتم سعی می کند فیلم هایی را که در لیست های موفق قبلی بوده اند، در ترکیب های جدید امتحان کند.

```

- Algorithm 1: Standard Harmony Search (Single-Objective)
- 1. Initialize Parameters: HMS, HMCR, PAR, MaxIterations.
- 2. Initialize HM: Generate HMS random solutions in the candidate space.
- 3. Evaluate: Calculate fitness  $f(x)$  for all solutions in HM.
- 4. While ( $t < \text{MaxIterations}$ ):
-     a. Improvise a new solution  $x_{\text{new}} = (x_1, x_2, \dots, x_K)$ :
-         For each dimension  $j = 1$  to  $K$ :
-             If  $\text{rand}(0,1) < \text{HMCR}$ :
-                  $x_j = \text{value from HM}[\text{random\_index}][j]$ 
-             If  $\text{rand}(0,1) < \text{PAR}$ :
-                  $x_j = \text{Adjust}(x_j)$  // Small change or neighbor swap
-         Else:
-              $x_j = \text{random\_item\_from\_candidates}()$ 
-     b. Selection/Update:
-         If  $f(x_{\text{new}})$  is better than  $f(x_{\text{worst}})$  in HM:
-             Replace  $x_{\text{worst}}$  with  $x_{\text{new}}$ .
-     c.  $t = t + 1$ .
- 5. Return: Best solution in HM.

```


- در **MA** : تمرکز بر این است که ابتدا یک ترکیب کلی خوب از آیتم‌ها پیدا شود (تقاطع) و سپس با جابجایی‌های کوچک (جستجوی محلی)، دقت یا تنوع آن لیست به حداکثر برسد. در رویکرد **لامارکی** (رایج در سیستم‌های توصیه‌گر)، هم مقدار برازندگی و هم ساختار راه حل (ژن‌ها) پس از جستجوی محلی به‌روزرسانی می‌شوند.

```

- Algorithm 2: Standard Memetic Algorithm (Single-Objective)
- 1. Initialize: Create an initial population of size N.
- 2. Evaluate: Calculate fitness  $f(x)$  for each individual.
- 3. While (Termination Condition not met):
-   a. Selection: Choose parents from the current population.
-   b. Recombination: Apply Crossover to parents to generate offspring  $P_{off}$ .
-   c. Mutation: Apply Mutation to  $P_{off}$  to generate  $P_{mut}$ .
-   d. Local Search (The Memetic Step):
-     For each individual  $x$  in  $P_{mut}$ :
-        $x_{refined} = \text{Local\_Search\_Procedure}(x)$ 
-       // e.g., greedy swaps to improve fitness
-        $x = x_{refined}$ 
-   e. Evaluation: Calculate  $f(x)$  for the refined individuals.
-   f. Survival: Select N individuals for the next generation (e.g., elitism).
- 4. Return: Best solution found.

```

۲- وظیفه دوم: تعریف بهینه‌سازی چند هدفه

در این بخش، مسئله انتخاب فهرست Top-K به عنوان یک مسئله بهینه‌سازی ترکیبیاتی با اهداف متضاد و قيود مشخص تعریف می‌شود. (برای هر دو الگوریتم)

نمادگذاری و داده‌های ورودی

- $\mathcal{U}$: مجموعه کاربران
- $\mathcal{I}$: مجموعه آیتم‌ها (فیلم‌ها)

$u \in \mathcal{U}$ برای هر کاربر ، اطلاعات زیر در اختیار است :

(۱) مجموعه آیتم‌های کاندید (آیتم‌هایی که کاربر قبلاً تعامل/مشاهده نداشته است) :

$$\mathcal{C}_u \subseteq \mathcal{I}$$

(۲) امتیاز پیش‌بینی‌شده‌ی مدل برای زوج (u, i) :

$$\hat{r}_{u,i} \in \mathbb{R}$$

(۳) اطلاعات ژانری هر آیتم به‌صورت یک بردار باینری :

$$\mathbf{g}_i \in \{0,1\}^G$$

متغیر تصمیم و فضای جستجو

برای هر کاربر یک راه‌حل، یک فهرست مرتب از طول K است :

$$\mathbf{x} = (i_1, i_2, \dots, i_K)$$

با شرایط زیر :

(۲) هر آیتم از کاندیدها انتخاب شود :

$$i_j \in \mathcal{C}_u \quad \forall j = 1, \dots, K$$

(۳) آیتم‌ها تکراری نباشند :

$$i_a \neq i_b \quad \forall a \neq b$$

(۴) ترتیب آیتم‌ها مهم است (رتبه‌بندی حفظ می‌شود) .

بنابراین فضای جستجو :

$$\Omega_u = \{(i_1, \dots, i_K) \mid i_j \in \mathcal{C}_u, i_a \neq i_b\}$$

توابع هدف

برای هر فهرست پیشنهادی برای کاربر، اهداف اصلی عبارت‌اند از :

- (Accuracy / Precision proxy) بیشینه‌سازی دقت/مرتبط‌بودن
- (Intra-List Diversity) بیشینه‌سازی تنوع درون فهرستی

هدف اول: دقت/مرتبط‌بودن

میانگین امتیاز پیش‌بینی شده (ساده و رایج) :

$$f_1(\mathbf{x}; u) = \frac{1}{K} \sum_{j=1}^K \hat{r}_{u,i_j}$$

هدف: $\max f_1(\mathbf{x}; u)$

معیار رتبه‌محور (پیشنهادی چون ترتیب مهم است) :

ابتدا مفهوم زیر را تعریف می‌کنیم :

$$\text{DCG}(\mathbf{x}; u) = \sum_{j=1}^K \frac{\hat{r}_{u,i_j}}{\log_2(j+1)}$$

سپس لیست ایده‌آلی با مرتب‌سازی کاندیدها بر اساس امتیاز ساخته می‌شود و :

$$\text{IDCG}(u) = \text{DCG}(\mathbf{x}^*; u)$$

در نهایت :

$$f_1^{\text{NDCG}}(\mathbf{x}; u) = \frac{\text{DCG}(\mathbf{x}; u)}{\text{IDCG}(u) + \varepsilon}$$

$\varepsilon > 0$ برای جلوگیری از تقسیم بر صفر است که هدف $\max f_1^{\text{NDCG}}(\mathbf{x}; u)$ است .

هدف دوم: تنوع درون فهرستی با فاصله Jaccard

برای هر آیتم مجموعه ژانرهای آن را تعریف می‌کنیم :

$$\Gamma(i) = \{t \in \{1, \dots, G\} \mid g_{i,t} = 1\}$$

شباهت جاکارد بین دو آیتم :

$$\text{JaccardSim}(i, j) = \frac{|\Gamma(i) \cap \Gamma(j)|}{|\Gamma(i) \cup \Gamma(j)| + \varepsilon}$$

فاصله (عدم‌شباهت) جاکارد :

$$d_J(i, j) = 1 - \text{JaccardSim}(i, j)$$

تنوع درون فهرستی :

$$\text{ILD}(\mathbf{x}) = \frac{2}{K(K-1)} \sum_{1 \leq p < q \leq K} d_J(i_p, i_q)$$

پس :

$$f_2(\mathbf{x}) = \text{ILD}(\mathbf{x})$$

هدف $\max f_2(\mathbf{x})$ است.

قید حداقل پوشش ژانری

برای جلوگیری از راه‌حل‌های بدیهی (مثلاً انتخاب آیتم‌های بسیار مشابه از یک ژانر)، قید زیر اعمال می‌شود .

ژانرهای پوشش داده‌شده توسط لیست :

$$\Gamma(\mathbf{x}) = \bigcup_{j=1}^K \Gamma(i_j)$$

تعداد ژانرهای متمایز در لیست :

$$GC(\mathbf{x}) = |\Gamma(\mathbf{x})|$$

قید حداقل پوشش :

$$GC(\mathbf{x}) \geq M$$

M یک مقدار ثابت (هایپرپارامتر) است.

روش اعمال قید

برای مدیریت قید حداقل پوشش چند روش استاندارد وجود دارد. باید یکی را انتخاب و شفاف گزارش کنیم.

ابتدا مقدار «نقض قید» را تعریف می‌کنیم :

$$v(\mathbf{x}) = \max(0, M - GC(\mathbf{x}))$$

روش (تعمیر راه حل)

اگر قید برقرار نبود، با جایگزینی تعدادی آیتم کاری می‌کنیم که ژانرهای جدید وارد لیست شوند ، تا شرط برقرار گردد .

ایده‌ی اجرایی: تا زمانی که قید برقرار نیست، یک ژانر پوشش داده نشده انتخاب می‌شود و آیتمی که آن ژانر را دارد (و در لیست نیست) جایگزین یکی از آیتم‌های کم‌اثر (مثلاً آیتم انتهایی لیست یا آیتم با امتیاز کمترین) می‌شود .

روش (تابع جریمه)

در صورت استفاده از ترکیب وزن دار اهداف، یک تابع تک هدفه جریمه دار می سازیم :

$$J(\mathbf{x}) = w_1 \tilde{f}_1(\mathbf{x}) + w_2 \tilde{f}_2(\mathbf{x}) - \lambda \tilde{v}(\mathbf{x})$$

که در آن :

$$\tilde{v}(\mathbf{x}) = \frac{v(\mathbf{x})}{M}$$

$$\lambda > 0 \text{ و } w_1 + w_2 = 1 \text{ و } w_1, w_2 \geq 0$$

روش (انتخاب مبتنی بر شدنی بودن)

در مقایسه‌ی دو راه حل :

- (۱) اگر فقط یکی شدنی باشد، پس بهتر است .
- (۲) اگر هر دو شدنی باشند، با معیار چندهدفه (پارتو یا وزن دار) مقایسه می شوند .
- (۳) اگر هر دو نشدنی باشند، آنکه نقض کمتری دارد بهتر است :

$$v(\mathbf{x}) < v(\mathbf{y}) \Rightarrow \mathbf{x} \text{ بهتر از } \mathbf{y}$$

روش (عملگرهای آگاه از قید)

در طراحی جهش/ترکیب/حرکت، انتخاب‌هایی انجام می دهیم که احتمال تولید لیست‌های ناقض قید کم شود؛ مثلاً هنگام جایگزینی آیتم، ترجیح دهیم آیتم جدید ژانر جدید اضافه کند .

نرمال سازی اهداف

برای مقایسه منصفانه و همچنین ترکیب اهداف (در روش وزن دار) لازم است اهداف نرمال شوند .

Min-Max روی جمعیت / حافظه

برای هدف (f) روی مجموعه مرجع (P) داریم: (Harmony Memory یا جمعیت)

$$f_m^{\min} = \min_{\mathbf{x} \in P} f_m(\mathbf{x}), \quad f_m^{\max} = \max_{\mathbf{x} \in P} f_m(\mathbf{x})$$

نرمال سازی :

$$\tilde{f}_m(\mathbf{x}) = \frac{f_m(\mathbf{x}) - f_m^{\min}}{f_m^{\max} - f_m^{\min} + \varepsilon}$$

$$\tilde{f}_m(\mathbf{x}) \in [0,1]$$

نرمال سازی ذاتی

اگر از هدف معیار رتبه محور استفاده شود، خود هدف دقت ذاتاً در بازه ۰ تا ۱ قرار می گیرد و همچنین تنوع درون فهرستی نیز معمولاً در بازه ی ۰ تا ۱ است .

مدیریت چند هدفه (ترکیب/انتخاب راه حل ها)

سه راهبرد استاندارد برای چندهدفه کردن مسئله وجود دارد .

غلبه پارتو - پیشنهاد اصلی

برای پیشینه سازی، می گوئیم یکی بر دیگری غالب است اگر :

$$\forall m: f_m(\mathbf{x}) \geq f_m(\mathbf{y}) \quad \text{و} \quad \exists m: f_m(\mathbf{x}) > f_m(\mathbf{y})$$

خروجی طبیعی این روش، مجموعه‌ی راه‌حل‌های **غیرمغلوب** (جبهه پارتو) است. برای حفظ تنوع روی جبهه می‌توان از معیارهایی استفاده کرد (مثل crowding distance).

ترکیب وزن دار

یک تابع تک‌هدفه از اهداف نرمال شده ساخته می‌شود

$$J(\mathbf{x}) = \sum_{m=1}^M w_m \tilde{f}_m(\mathbf{x})$$

$$\sum_m w_m = 1, w_m \geq 0$$

روش قید ϵ

یکی از اهداف را بهینه و بقیه را قید می‌کنیم. مثال :

$$\max f_1(\mathbf{x}; u) \quad \text{s.t.} \quad f_2(\mathbf{x}) \geq \epsilon, \text{ GC}(\mathbf{x}) \geq M$$

با تغییر اپسیلون نقاط مختلف trade-off تولید می‌شوند .

(Bias / Fairness / Novelty) - اهداف اضافی پیشنهادی

به جز دقت و تنوع، اهداف دیگری نیز می‌توان اضافه کرد. هدف‌های زیر در سیستم‌های توصیه‌گر بسیار رایج و قابل دفاع هستند .

هدف اضافه ۱: کاهش بایاس محبوبیت / افزایش Novelty

محبوبیت یک آیتم در داده‌ی آموزش :

$$\text{pop}(i) = |\{(u, i) \in \mathcal{D}_{train}\}|$$

(الف) کمینه‌سازی جریمه محبوبیت

$$f_3^{\text{pop}}(\mathbf{x}) = \frac{1}{K} \sum_{j=1}^K \log(1 + \text{pop}(i_j))$$

هدف : $\min f_3^{\text{pop}}$

(ب) بیشینه‌سازی Novelty

$$f_3^{\text{nov}}(\mathbf{x}) = \frac{1}{K} \sum_{j=1}^K [-\log(1 + \text{pop}(i_j))]$$

هدف : $\max f_3^{\text{nov}}$

هدف اضافه ۲: عدالت/کالیبراسیون ژانری

برای اینکه توزیع ژانری توصیه‌ها خیلی نامتوازن نباشد یا با سلیقه‌ی کاربر هم‌راستا شود ، می‌توان فاصله‌ی بین توزیع‌ها را کمینه کرد .

ابتدا وزن رتبه به سبک DCG :

$$w_j = \frac{1}{\log_2(j+1)}$$

نمایش ژانر در لیست :

$$E_t(\mathbf{x}) = \sum_{j=1}^K w_j g_{i_j,t}$$

توزیع ژانری لیست :

$$q_{\mathbf{x}}(t) = \frac{E_t(\mathbf{x})}{\sum_{\ell=1}^G E_{\ell}(\mathbf{x}) + \varepsilon}$$

نسبت به توزیع هدف یکنواخت | **Fair Exposure** (الف)

$$p_{\text{target}}(t) = \frac{1}{G}$$

نسبت به پروفایل ژانری کاربر | **Calibration** (ب)

اگر تاریخچه کاربر در داده‌ی آموزش :

$$\mathcal{H}_u = \{i \mid (u, i) \in \mathcal{D}_{\text{train}}\}$$

توزیع ژانری کاربر :

$$p_u(t) = \frac{\sum_{i \in \mathcal{H}_u} g_{i,t}}{\sum_{\ell=1}^G \sum_{i \in \mathcal{H}_u} g_{i,\ell} + \varepsilon}$$

فاصله **Jensen–Shannon** :

ابتدا :

$$m(t) = \frac{1}{2}(q_{\mathbf{x}}(t) + p(t))$$

سپس :

$$D_{JS}(q_x \parallel p) = \frac{1}{2} \sum_{t=1}^G q_x(t) \log \frac{q_x(t)}{m(t)} + \frac{1}{2} \sum_{t=1}^G p(t) \log \frac{p(t)}{m(t)}$$

می‌توان هدف را به یکی از دو شکل تعریف کرد :

- کمینه‌سازی فاصله :

$$\min D_{JS}(q_x \parallel p)$$

یا

- بیشینه‌سازی نسخه منفی فاصله :

$$f_4(\mathbf{x}; u) = -D_{JS}(q_x \parallel p)$$

پس صورت نهایی مسئله چندهدفه برای هر کاربر :

$$\max_{\mathbf{x} \in \Omega_u} \mathbf{F}(\mathbf{x}; u) = \max_{\mathbf{x} \in \Omega_u} (f_1(\mathbf{x}; u), f_2(\mathbf{x}), f_3(\mathbf{x}), f_4(\mathbf{x}; u))$$

به‌طوری‌که :

$$GC(\mathbf{x}) \geq M$$

نکته‌ی اجرایی برای استفاده در MA و HSA :

- در هر دو الگوریتم، یک راه‌حل برابر یک لیست است .
- برای هر راه‌حل باید بردار اهداف محاسبه شود .
- قید پوشش ژانری باید با یکی از روش‌های گفته‌شده، اعمال شود .
- برای چندهدفه کردن، پیشنهاد اصلی استفاده از **پارتو** است؛ اما روش وزن‌دار نیز قابل استفاده است .

۳- وظیفه سوم: پیاده‌سازی و تحلیل نتایج

در سیستم‌های توصیه‌گر مدرن، تمرکز صرف بر دقت می‌تواند منجر به پیشنهادهای یکنواخت و تکراری شود که رضایت کاربر در بلندمدت را کاهش می‌دهد. به همین دلیل در این مسئله، هدف تولید لیست توصیه‌ی **Top-K** برای هر کاربر با در نظر گرفتن دو هدف متضاد است: یکی **بیشینه‌سازی دقت** پیشنهادهای و دیگری **بیشینه‌سازی تنوع درون لیست**. همچنین یک **قید پوشش ژانری** تعریف شده که طبق آن هر لیست باید حداقل شامل **M ژانر متمایز** باشد. این مشخصات باعث می‌شوند مسئله به طور طبیعی یک **مسئله بهینه‌سازی چندهدفه** باشد.

برای هر کاربر مجموعه‌ای از آیت‌های **کاندید** (آیت‌هایی که کاربر قبلاً ندیده است) به همراه امتیاز **پیش‌بینی‌شده** هر آیت (تولیدشده توسط مدل **MF** پایه) در اختیار داریم. فضای جستجو تمامی ترکیب‌های ممکن از **K** آیت یکتاست که می‌توان به آن کاربر توصیه کرد. هر ترکیب یک **راه‌حل** بالقوه است و دارای دو **تابع هدف** می‌باشد:

- **f₁ : دقت لیست توصیه** که می‌تواند مثلاً مجموع یا میانگین امتیازات پیش‌بینی‌شده‌ی مدل **MF** برای آیت‌های انتخاب‌شده باشد (مدل **MF** به عنوان منبع امتیازدهی و مبنای محاسبه دقت به کار می‌رود). هرچه امتیازات پیش‌بینی‌شده آیت‌های لیست بالاتر باشند، لیست دقیق‌تر تلقی می‌شود.

- **f₂ : تنوع درون لیست** که میزان تفاوت آیت‌های لیست را نشان می‌دهد. **تنوع (ILD)** معمولاً با میانگین فاصله زوجی بین آیت‌های لیست اندازه‌گیری می‌شود. برای مثال می‌توان ژانر فیلم‌ها را به صورت بردار ویژگی (بردار باینری ژانرها) نمایش داد و سپس **فاصله کسینوسی** یا **Jaccard** را بین بردارهای هر دو فیلم در لیست محاسبه و میانگین‌گیری کرد. نتیجه ۰ به معنای کاملاً مشابه بودن آیت‌ها (تنوع صفر) و ۱ به معنای حداکثر تنوع (کاملاً متفاوت بودن همه آیت‌ها) است. به بیان ساده، **تنوع درون لیست معیاری از متفاوت بودن آیت‌های یک لیست توصیه است**.

با توجه به دوهدفه بودن مسئله، راه حل بهینه یکتایی وجود ندارد؛ بلکه مجموعه‌ای از راه حل‌های غیرمغلوب (Pareto-optimal) وجود دارد که جبهه پارتو را تشکیل می‌دهند. هر راه حل پارتو نشان‌دهنده یک توازن ممکن بین دقت و تنوع است که نمی‌توان یکی از اهداف را بهبود داد مگر به بهای تضعیف هدف دیگر. در ادامه، دو الگوریتم تکاملی/فراابتکاری برای حل این مسئله تشریح می‌شوند. همچنین دو رویکرد برای مدیریت چندهدفه بودن (روش پارتو و روش ترکیب وزن دار) در پیاده‌سازی این الگوریتم‌ها استفاده خواهد شد.

در ابتدای پیاده‌سازی، لازم است داده‌های ۱۰۰K MovieLens بارگذاری و پیش‌پردازش شوند. فایل‌های اصلی عبارت‌اند از:

- **u.data**: شامل ۱۰۰,۰۰۰ امتیازدهی کاربران به فیلم‌ها
- **u.item**: شامل اطلاعات فیلم‌ها (عنوان فیلم، سال، و مجموعه‌ای از ۱۹ ستون باینری مربوط به ژانرهای فیلم). از این فایل می‌توان ژانرهای هر فیلم را استخراج کرد. ژانرها به صورت بردار طول ۱۹ نشان داده می‌شوند که هر بعد نمایانگر یک ژانر است.
- **u.genre**: نگاشت نام ژانرها به شناسه عددی (صرفاً جهت اطلاع از نام ژانرها و تفسیر نتایج به کار می‌رود).

پس از بارگذاری، داده‌ها باید به شکل مناسبی سازمان‌دهی شوند. برای هر کاربر:

۱. لیست آیتم‌های دیده‌شده توسط کاربر از روی u.data مشخص می‌شود.
۲. لیست آیتم‌های کاندید که کاربر هنوز ندیده است (تمام آیتم‌های دیتاست منهای آیتم‌های دیده‌شده). این مجموعه کاندید ورودی الگوریتم بهینه‌سازی است. می‌توان برای کارایی بیشتر، تعداد کاندیدها را محدود کرد (مثلاً ۱۰۰ آیتم با بالاترین امتیاز پیش‌بینی‌شده) تا فضای جستجو کوچکتر شود.
۳. امتیاز پیش‌بینی‌شده MF برای هر آیتم کاندید: مدل MF از پیش آموزش‌یافته یا آموزش داده‌شده روی u.data برای هر کاربر و هر آیتم یک امتیاز علاقه تولید می‌کند. این

امتیازها مبنای محاسبه هدف دقت هستند. در صورت عدم دسترسی به مدل از پیش آموزش یافته، می توان یک مدل MF ساده (مثلاً با کتابخانه Surprise یا SGD) روی داده ی آموزش ساخت. اما طبق صورت مسئله، مدل MF آماده فراهم است و نیازی به بهبود آن نیست.

پس از آماده سازی موارد فوق، می توانیم معیارهای ارزیابی را تعریف کنیم:

- **دقت لیست Top-K** برای یک کاربر: از آنجا که امتیاز MF نشان دهنده میزان مطلوبیت آیتم برای کاربر است، دقت را می توان میانگین یا مجموع امتیازات MF برای آیتم های انتخاب شده تعریف کرد. به عنوان جایگزین، اگر مجموعه ای از آیتم های واقعاً پسندیده شده توسط کاربر (مثلاً امتیاز واقعی ≥ 4) در داده های تست موجود باشد، می توان **Precision@K** یا **Recall@K** را نیز محاسبه کرد. اما در این تمرین، از امتیاز MF به عنوان معیار مستقیم دقت استفاده می شود.

- **تنوع درون لیست (ILD)**: طبق تعریف، ابتدا هر آیتم را به صورت بردار ویژگی نمایش می دهیم (در اینجا بردار ژانری ۱۹ بعدی). سپس برای هر جفت آیتم در لیست فاصله بین دو بردار را محاسبه می کنیم (مثلاً فاصله کسینوسی یا Jaccard). **تنوع برابر میانگین این فواصل زوجی** است. مقدار ۱ نشان دهنده حداکثر تنوع (آیتم ها کاملاً متفاوت) و ۰ نشان دهنده حداقل تنوع (آیتم ها یکسان) است. یک معیار ساده تر تنوع می تواند **تعداد ژانرهای منحصربه فرد در لیست** نیز باشد. هرچه تعداد ژانرهای مختلف بیشتر باشد، تنوع بیشتر است؛ به خصوص این معیار برای بررسی **قید پوشش ژانری** مستقیم تر است (مثلاً لیست باید $M \geq$ ژانر داشته باشد). در ارزیابی نهایی می توان هر دو نوع تنوع (عددی و پوشش ژانر) را گزارش کرد.

- **پوشش ژانری (قید)**: درصد یا تعداد ژانرهای حاضر در لیست نسبت به کل ژانرهای ممکن. این معیار باید حداقل M باشد (مثلاً اگر $M=3$ ، لیست باید حداقل ۳ ژانر مختلف داشته باشد وگرنه مردود است).

- طراحی نوتبوک Google Colab با پشتیبانی از اجرای کامل بر روی دیتاست MovieLens ۱۰۰K
- پیاده‌سازی کامل هر دو الگوریتم به صورت چندهدفه با استفاده از روش پارتو و ترکیب وزن‌دار
- اعمال قید پوشش ژانری
- ترسیم نمودار جبهه پارتو، نمودار پیشرفت و تحلیل trade-off
- زمان‌سنجی اجرا و مقایسه نهایی

الگوریتم جستجوی هارمونی (HSA)

روش فراابتکاری الهام گرفته از فرآیند بداهه‌نوازی در موسیقی است. در این استعاره، هر راه‌حل مانند یک **ملودی** (هارمونی) و هر **متغیر تصمیم** مانند یک **نت موسیقی** در آن ملودی است. نوازندگان با تنظیم مکرر زیروبمی (pitch) سازهای خود به یک ترکیب هماهنگ می‌رسند؛ به‌طور مشابه الگوریتم HSA با **تنظیم مقادیر متغیرها** در راه‌حل‌ها به یک جواب بهینه نزدیک می‌شود. از مزایای این الگوریتم سادگی پیاده‌سازی، تعداد کم پارامترهای تنظیم و توانایی موازنه جستجوی گسترده/محدود است.

در بافت مسئله ما:

- هر **راه‌حل** همان **لیست Top-K** توصیه به یک کاربر است. می‌توان آن را به صورت یک آرایه با طول K از شناسه آیتم‌ها نمایش داد (هر عنصر معرف یک آیتم منتخب در لیست). ترتیب آیتم‌ها نیز ممکن است مهم باشد (اگر قصد داریم لیست را رتبه‌بندی شده ارائه کنیم)، بنابراین نمایش می‌تواند یک ترتیب مرتب‌شده از K آیتم یکتا باشد.
- هر **متغیر تصمیم** متناظر با یکی از **جایگاه‌های لیست** است (مثلاً پوزیشن ۱ تا K). مقدار آن یک آیتم از مجموعه کاندید است که در آن جایگاه قرار می‌گیرد.

• یک هارمونی (Harmony) در الگوریتم معادل یک لیست توصیه پیشنهادی است. کیفیت (fitness) آن توسط دو تابع هدف (دقت و تنوع) سنجیده می‌شود. در نسخه تک‌هدفه (مثلاً هنگام استفاده از ترکیب وزن‌دار اهداف)، می‌توان یک اسکالر براساس این دو هدف محاسبه کرد. در نسخه چندهدفه (پارتو)، کیفیت به صورت برداری ارزیابی می‌شود.

الگوریتم HSA یک حافظه هارمونی (Harmony Memory) نگهداری می‌کند که شامل تعداد ثابتی (HMS) راه‌حل (ملودی) جاری است. سپس به صورت تکراری راه‌حل‌های جدید تولید و جایگزین راه‌حل‌های ضعیف حافظه می‌شوند تا به تدریج کیفیت بهبود یابد. مراحل اصلی به شرح زیر است:

۱. مقداردهی اولیه: تعدادی HMS راه‌حل اولیه به صورت تصادفی تولید می‌کنیم (انتخاب K آیتم تصادفی از بین کاندیدها برای هر هارمونی) و معیارهای دقت و تنوع را برای آن‌ها محاسبه می‌کنیم. همچنین می‌توان برخی راه‌حل‌های اولیه را به شکل حریصانه تولید کرد (مثلاً چند لیست که شامل آیتم‌های با بالاترین امتیاز MF هستند برای داشتن دقت بالا، و چند لیست متنوع برای داشتن تنوع بالا) تا تنوع اولیه جمعیت افزایش یابد.

۲. تولید هارمونی جدید: در هر تکرار، الگوریتم یک راه‌حل جدید می‌سازد. این کار برای هر کدام از K موقعیت آیتم در لیست انجام می‌شود:

○ با احتمال HMCR، از حافظه هارمونی انتخاب می‌کنیم: بدین صورت که یکی از راه‌حل‌های فعلی در حافظه و سپس یکی از آیتم‌های آن راه‌حل در جایگاه جاری را انتخاب کرده و در راه‌حل جدید قرار می‌دهیم. ایده این است که از خاطره جمعی راه‌حل‌های خوب قبلی استفاده کنیم.

○ با احتمال $(1 - HMCR)$ ، انتخاب تصادفی می‌کنیم: یک آیتم تصادفی از بین کل آیتم‌های کاندید (یا از بین آیتم‌هایی که تاکنون در راه‌حل جدید نیامده‌اند) برای این جایگاه انتخاب می‌شود. این مرحله برای تزریق تنوع جدید است.

- پس از انتخاب مقدار اولیه برای آن متغیر، با احتمال **PAR** یک **تنظیم جزئی** (**Pitch Adjustment**) انجام می‌شود. در مسائل عددی، این به معنی تغییر مختصر مقدار انتخابی (مثلاً افزودن یک مقدار کوچک به متغیر) است. در این مسئله گسسته می‌توان **Pitch Adjustment** را به صورت انتخاب یک آیتم نزدیک به آیتم فعلی تعریف کرد. برای مثال: اگر آیتم انتخاب شده فیلمی از ژانر کمدی است و هدف تنوع است، شاید با یک احتمال کوچک آن را با فیلم دیگری از ژانر کمتر تکرارشونده جایگزین کنیم. یا اگر هدف دقت است، آیتم را با یک آیتم دیگر با امتیاز **MF** کمی پایین‌تر یا بالاتر جایگزین کنیم. این تنظیم در فضای گسسته معادل نوعی **جهش محلی** است که کمک می‌کند اندکی ترکیب تغییر کند.

- طی این فرایند باید مراقب **یکتا بودن آیتم‌ها** در لیست بود؛ اگر انتخاب حافظه یا انتخاب تصادفی منجر به قراردادن آیتمی شود که قبلاً در همین لیست جدید آمده، باید آیتم دیگری انتخاب یا جایگزین کنیم (می‌توان مکانیزم جلوگیری از تکرار را در همان انتخاب یا در پایان با یک مرحله تعمیر انجام داد).

۳. **اعمال قید پوشش ژانر:** راه‌حل جدید ممکن است قید **M** ژانر را نقض کند (مثلاً تنها ۱ یا ۲ ژانر در آن باشد). برای رفع این موضوع دو رویکرد کلی وجود دارد:

- **مکانیزم تعمیر (Repair):** پس از ساخت راه‌حل، ژانرهای موجود در آن را می‌شماریم. اگر تعداد ژانرهای یکتا $M <$ بود، به تعداد لازم آیتم‌های فعلی را با آیتم‌هایی از ژانرهای غایب جایگزین می‌کنیم تا قید تأمین شود. یک راه این است که ژانرهای کم‌نمایش لیست شناسایی شوند و آیتم‌هایی از ژانرهای جدید (که هنوز در لیست نیستند) که امتیاز پیش‌بینی مناسبی دارند اضافه شوند. این کار نوعی **اصلاح پس‌پردازشی** است که راه‌حل را به نزدیک‌ترین حالت شدنی تبدیل می‌کند.

- **روش مبتنی بر شدنی بودن:** در فرایند تولید (مرحله ۲) سعی می‌کنیم اصلاً راه‌حل نامولد تولید نشود. مثلاً هنگام افزودن هر آیتم، احتمال انتخاب آیتم از ژانری جدید

را افزایش دهیم یا اگر تاکنون ژانر خاصی انتخاب نشده، در یک جایگاه آن ژانر را الزامی کنیم. یا می‌توان در مرحله انتخاب از حافظه و انتخاب تصادفی کاری کرد که راه‌حل موقت همیشه M ژانر داشته باشد (هر چند این پیچیده‌تر است). همچنین می‌توان راه‌حل‌های نامعتبر را رد کرد و تولید Harmony را تا تأمین قید ادامه داد، یا هنگام ارزیابی جریمه سنگینی برای نقض قید در نظر گرفت که احتمال انتخاب شدن آن را به صفر میل دهد.

۴. ارزیابی و به‌روزرسانی حافظه: پس از ساخت راه‌حل جدید و (احتمالاً) تعمیر آن، مقادیر اهداف f_1 و f_2 آن محاسبه می‌شود. در حالت تک‌هدفه وزن‌دار، ترکیب وزنی $F = w \times f_1 + (1 - w) \times f_2$ محاسبه و به عنوان fitness استفاده می‌شود. در حالت چندهدفه پارتو، مقایسه براساس غالب/مغلوب بودن انجام می‌شود (جزئیات در بخش بعد). الگوریتم HSA کلاسیک معمولاً هر بار جدیدترین راه‌حل را با بدترین راه‌حل موجود در حافظه مقایسه و در صورت بهتر بودن جایگزین می‌کند. در نسخه چندهدفه، این مفهوم جایگزینی می‌تواند براساس غلبه پارتو باشد: اگر راه‌حل جدید توسط هیچ راه‌حل دیگری در حافظه مغلوب نشود، می‌توان آن را اضافه کرد. در صورت پر شدن ظرفیت، می‌توان راه‌حل‌های تحت غلبه یا راه‌حل‌های ضعیف (مثلاً دارای کمترین تنوع و دقت) را حذف کرد. همچنین می‌توان از آرشیو خارجی برای نگهداری راه‌حل‌های غیرمغلوب استفاده کرد و حافظه هارمونی را جداگانه برای تولید نسل بعد به‌روز کرد.

۵. تکرار مراحل تولید تا پایان معیار توقف: الگوریتم به تولید راه‌حل‌های جدید ادامه می‌دهد تا مثلاً به تعداد تکرار مشخص (max iterations) یا عدم بهبود طولانی (convergence) برسد. خروجی الگوریتم در حالت وزن‌دار، بهترین راه‌حل نهایی (یا مجموعه‌ای از بهترین‌ها در طول اجرا) و در حالت پارتو، مجموعه راه‌حل‌های غیرمغلوب به‌دست‌آمده طی اجرا است.

تنظیم پارامترها: پارامترهای کلیدی HSA عبارت‌اند از سایز حافظه، نرخ استفاده از حافظه که معمولاً بین ۰.۷ تا ۰.۹۵ است، نرخ تنظیم زیروبم که معمولاً کوچک مثلاً ۰.۱ تا ۰.۳ است، دامنه یا احتمال تغییر در pitch. همچنین K (طول لیست) و M (حداقل ژانر) جزو پارامترهای مسئله هستند. این مقادیر را می‌توان با آزمون و خطا یا بر اساس توصیه‌های ادبیات تنظیم کرد اما در اینجا تمامی دانشجویان باید پارامترهای یکسانی انتخاب که مقایسه عادلانه باشد.

الگوریتم Harmony Search به دلیل سادگی نسبی و استفاده از **ترکیب حافظه و جستجوی تصادفی**، در بسیاری مسائل به نتایج رقابتی با الگوریتم‌های معروفی چون GA و PSO دست یافته است. با این حال، ممکن است دچار همگرایی زودرس شود یا دقت بهینه پایین‌تری نسبت به روش‌های ترکیبی پیشرفته‌تر داشته باشد. در اینجا ما HSA را به عنوان یک رویکرد ابتکاری برای بهینه‌سازی همزمان دقت/تنوع به کار می‌گیریم. در ادامه الگوریتم ممیتیک که یک رویکرد قوی‌تر ترکیبی است معرفی می‌شود.

الگوریتم ممیتیک (MA)

الگوریتم ممیتیک (MA) یک روش تکاملی پیشرفته است که **جستجوی تکاملی جمعیتی** مثل GA را با **بهبود محلی ترکیب** می‌کند. در واقع می‌توان آن را **ژنتیک ترکیب‌شده با صعود تپه** یا **الگوریتم ژنتیک با یادگیری فردی** قلمداد کرد. ایده اصلی این است که هر راه‌حل (فرد) پس از تولید شدن توسط عملگرهای تصادفی، به کمک یک رویه‌ی **جستجوی محلی** موسوم به *meme* در استعاره‌ی ممیتیک **اصلاح و بهبود** یابد. این کار منجر به تسریع همگرایی و رسیدن به راه‌حل‌های با کیفیت‌تر می‌شود، چرا که کاوش سراسری GA با بهره‌گیری از دانش محلی هر راه‌حل همراه می‌گردد. به بیان دیگر، **الگوریتم ممیتیک جستجوی کلی جمعیتی را با جستجوی موضعی ترکیب می‌کند تا به همگرایی سریع‌تر و راه‌حل‌های بهتر برسد**. بسیاری از پژوهش‌ها نشان داده‌اند که الگوریتم‌های ممیتیک نسبت به الگوریتم‌های تکاملی استاندارد کیفیت بالاتری از راه‌حل‌ها را با کارایی بهتر حاصل می‌کنند.

در زمینه مسئله ما، MA را می‌توان این‌گونه پیاده‌سازی کرد:

۱. **نمایش راه‌حل و جمعیت اولیه:** مشابه GA، یک جمعیت اولیه از N راه‌حل (لیست Top-K) به صورت تصادفی یا نیمه‌حریصانه تولید می‌شود. نمایش هر فرد همان آرایه K تایی آیت‌ها است (مانند قبل). باید قید یکتا بودن آیت‌ها رعایت شود. راه‌حل‌های اولیه را می‌توان طوری ایجاد کرد که تنوع جمعیت تضمین شود (مثلاً برخی با ژانرهای مختلف، برخی با آیت‌های محبوب‌تر).

۲. **ارزیابی اولیه:** مقدار اهداف f_1 و f_2 برای هر فرد محاسبه و در صورت وزن‌دار بودن، مقدار برازش ترکیبی تعیین می‌شود. همچنین می‌توان **راه‌حل‌های شدنی/ناشدنی** را در نظر گرفت: اگر فردی قید ژانر را ارضا نکند، یا حذف شود یا به آن جریمه تعلق گیرد یا تعمیر شود (انتخاب راهبرد با ماست).

۳. **عملگر انتخاب (Selection):** مانند GA، برای تولید نسل بعد، چند والد از بین جمعیت فعلی انتخاب می‌شوند. روش‌های معمول شامل **تورنامنت** یا **رولت‌شانسی متناسب با برازش** یا **NSGA-II selection** (در حالت چندهدفه مبتنی بر رتبه پارتو و فاصله ازدحام) هستند. هدف این است که راه‌حل‌های بهتر شانس بیشتری برای تولید فرزند داشته باشند، در عین حال تنوع نیز حفظ شود. در حالت چندهدفه پارتو، می‌توان از الگوریتم‌هایی مثل NSGA-II یا SPEA۲ برای انتخاب والدین بهره گرفت که مجموعه‌ای از راه‌حل‌های پارتو-بهینه را در هر نسل حفظ می‌کنند.

۴. **عملگر ترکیب (Crossover):** دو والد انتخاب‌شده **تبادل** اطلاعات می‌کنند تا فرزندان جدید ایجاد شوند. برای لیست Top-K می‌توان کراس‌اور را به صورت **تک‌نقطه‌ای** یا **دودویی** اعمال کرد: مثلاً یک نقطه تصادفی در آرایه انتخاب و بخش اول از والد A و بخش دوم از والد B ترکیب شوند. یا روش دیگر: **کراس‌اور مبادله‌ای** که در آن یک زیرمجموعه از آیت‌های والد A و بقیه از والد B گرفته شوند (با مراقبت از یکتا بودن آیت‌ها). نیاز است مکانیزمی برای برخورد با آیت‌های تکراری یا کمبود آیت در لیست فرزند در نظر بگیریم –

برای مثال، اگر آیتمی دوبار آمد، یکی را با یک آیتم از مجموعه والد دیگر که حضور ندارد جایگزین کنیم. هدف این است که فرزندان ویژگی‌های والدین را به ارث ببرند: مثلاً ترکیبی از آیتم‌های پربازده والد A و آیتم‌های متنوع والد B.

۵. **عملگر جهش (Mutation):** روی هر فرزند حاصل، یک جهش تصادفی کوچک اعمال می‌شود تا تنوع حفظ شود. جهش می‌تواند به صورت **تعویض یک آیتم** از لیست با یک آیتم دیگر از کاندیدها که در لیست نیست باشد. به احتمال کم، برای هر موقعیت در لیست یک آیتم جدید (تصادفی یا بر اساس الگوریتم کمکی) قرار می‌دهیم. یا می‌توان **دو آیتم درون لیست را جابجا کرد** (اگر ترتیب مهم است). جهش کمک می‌کند نواحی جدیدی از فضای راه‌حل کشف شود و از گیر افتادن در بهینه محلی جلوگیری شود.

۶. **اعمال قید و فیلتر:** هر فرزند تولیدشده را از نظر پوشش ژانری بررسی می‌کنیم. مشابه قبل، اگر قید را نقض کرده باشد، یا در همین مرحله **ترمیم** می‌کنیم (مثلاً جایگزینی برخی آیتم‌ها با ژانرهای جدید)، یا به جمعیت اضافه نمی‌کنیم (راه‌حل را دور می‌ریزیم و شاید جهش دیگری امتحان کنیم)، یا جریمه‌ای در ارزیابی می‌دهیم. روش انتخابی باید شفاف باشد؛ در این تمرین توصیه شده از **Repair** یا **انتخاب شدنی** استفاده شود.

۷. **بهبود محلی (Local Search):** این مرحله کلیدی ممیتیک است. روی هر فرد (راه‌حل) منتخب (یا همه افراد، یا بخشی از افراد برتر) یک الگوریتم بهبود محلی اعمال می‌شود. برای مسئله ما، یک بهبود محلی ساده می‌تواند چنین باشد:

○ **جابه‌جایی آیتم‌ها:** تک‌تک آیتم‌های لیست را امتحان کنیم که اگر آن را با یک آیتم خارج از لیست (از کاندیدهای باقیمانده) عوض کنیم آیا یکی از اهداف بهبود چشمگیری می‌یابد بدون اینکه دیگری خیلی افت کند؟ مثلاً یک آیتم بسیار تکراری ژانری را با آیتمی از ژانر متفاوت جایگزین کنیم تا تنوع بالا رود، مشروط بر اینکه افت دقت زیاد نباشد. یا برعکس، آیتم خیلی کم‌امتیاز را با آیتم امتیازبالای خارج

لیست جابجا کنیم تا دقت بالا رود، به شرط حفظ تنوع پایه. این شبیه یک حرکت یک نقطه‌ای در همسایگی راه حل است.

○ **تبادل دو آیتم:** اگر ترتیب مهم است، می‌توان جابجایی دو آیتم درون لیست را نیز آزمود تا شاید اثر رتبه‌بندی را بسنجد (مثلاً اگر تنوع به ترتیب حساس بود).

○ **افزودن آیتم ژانر جدید:** اگر هنوز کمتر از M ژانر دارد، حتماً یکی از آیتم‌های تکراری ژانر را با آیتم یک ژانر جدید عوض کنیم (این در واقع همان Repair است که می‌توان آن را بخشی از بهبود محلی دید).

○ **جستجوی محلی هدایت‌شده توسط وزن:** در روش ترکیب وزن دار، می‌توان تابع هدف وزن دار را به عنوان تابع هزینه در یک **Hill Climbing** به کار برد: بطوریکه همسایگان یک لیست (مثلاً لیست‌های با یک تغییر جزئی) را ارزیابی و اگر بهبود داشت جایگزین کنیم، تا به بهبود موضعی برسیم.

این بهبود محلی می‌تواند به صورت **Lamarckian** اعمال شود (تغییر مستقیم ژنوم فرد به حالت بهبودیافته) یا **Baldwinian** (تنها ارزش برآزش فرد را اصلاح کنیم بدون تغییر ژنوتیپ). معمولاً در ممیتیک‌ها Lamarckian استفاده می‌شود یعنی پس از بهبود، راه حل اصلاح‌شده مستقیماً جایگزین فرد قبلی می‌شود. لازم است تصمیم بگیریم همه فرزندان یا فقط بهترین‌ها تحت **local search** قرار گیرند و با چه شدتی (مثلاً تعداد حرکت‌های محلی). این تنظیمات بر سرعت و کیفیت تاثیر می‌گذارند. در تمرین حاضر، می‌توان برای سادگی روی هر فرزند یک یا دو حرکت بهبود انجام داد.

۸. **تشکیل نسل جدید:** اکنون تمام فرزندان (بهبودیافته) تشکیل جمعیت نسل بعد را می‌دهند. بسته به الگوریتم می‌توان **Elitism** داشت: مثلاً چند فرد برتر نسل قبل (یا پارتو-بهبوده‌های نسل قبل) را نیز به نسل جدید منتقل کنیم تا از دست نروند. در غیر این صورت، نسل قبل کاملاً با نسل جدید جایگزین می‌شود. جمعیت جدید سپس برای تکرار بعدی الگوریتم آماده است.

۹. **معیار توقف:** روند انتخاب-ترکیب-جهش-بهبود تا رسیدن به شرط توقف ادامه می‌یابد.

شرط می‌تواند تعداد نسل معین (مثلاً ۱۰۰ نسل)، یا رسیدن به بهبودی ناچیز در چند نسل متوالی، یا محدودیت زمانی باشد. در پایان، مشابه GA، ما یک مجموعه از بهترین راه‌حل‌ها داریم. در حالت وزن‌دار، معمولاً بهترین فرد آخر (یا کل جمعیت آخر که شاید همگی نزدیک به بهینه هستند) خروجی است. در حالت چندهدفه، همه راه‌حل‌های پارتو-بهینه نهایی را می‌توان خروجی گرفت (مثلاً تمامی افراد نسل آخر که هیچ‌کدام دیگری را مغلوب نمی‌کند).

در پیاده‌سازی هر دو الگوریتم فوق HSA و MA، باید نحوه‌ی در نظر گرفتن دو هدف مشخص شود. دو رویکرد متداول عبارت‌اند از:

- **بهینه‌سازی چندهدفه مبتنی بر پارتو (Pareto dominance):** در این شیوه، اهداف

به صورت جداگانه حفظ می‌شوند و مقایسه راه‌حل‌ها با مفهوم غالب/مغلوب انجام می‌شود.

راه‌حلی گفته می‌شود A بر B غالب است که $f_1(A) \geq f_1(B)$ و $f_2(A) \geq f_2(B)$ (بدون

کاستی در هیچ هدف)، و حداقل در یکی از اهداف بهتر باشد. الگوریتم باید تلاش کند

مجموعه‌ای از راه‌حل‌های ناب‌تری پذیر یا ناب‌ر **dominant** بیابد که نمایانگر جبهه پارتوی

مطلوب هستند. برای این منظور می‌توان از مکانیزم‌های انتخاب چندهدفه استفاده کرد؛ به

طور مثال در الگوریتم Memetic می‌توان NSGA-II را در انتخاب به کار گرفت که شامل

مرتب‌سازی غیربرتری (**Non-dominated sorting**) و محاسبه فاصله ازدحام

(**Crowding distance**) برای حفظ تنوع راه‌حل‌های پارتو است. در الگوریتم HSA هم

می‌توان حافظه هارمونی را طوری مدیریت کرد که همواره راه‌حل‌های غیرمغلوب حفظ شوند

(مثلاً یک آرشیو خارجی از پارتوست). هنگام جایگزینی در HSA، به جای مقایسه ساده

برآزش، باید چک شود آیا هارمونی جدید کسی را مغلوب می‌کند یا مغلوب کسی می‌شود؛

سپس بر اساس آن حافظه به‌روز شود. هدف این است که در پایان الگوریتم، مجموعه‌ای از

راه‌حل‌های مختلف با توازن‌های گوناگون دقت/تنوع به‌دست آید. برای تصمیم‌گیری در

انتخاب والدین یا حذف افراد اضافه، رتبه پارتو (چندجبهه‌ای) و شاید یک معیار تنوع

استفاده می‌شود. مثلاً در MA می‌توان ترکیبی از NSGA-II/SPEA۲ استفاده کرد. در

HSA، هر هارمونی جدید اگر غیر برتری باشد وارد حافظه می شود و هر هارمونی موجود که توسط دیگران مغلوب شد حذف می گردد؛ همچنین ممکن است نیاز باشد برای جلوگیری از رشد بیش از حد حافظه، از بین راه حل های غیر مغلوب قدیمی، آنهایی که از نظر تنوع شبیه دیگران هستند یا ارزش کمتری دارند حذف شوند.

- **بهینه سازی تک هدفه با تابع وزن دار:** در این روش، دو هدف را با ضرایب دلخواه ترکیب کرده و یک هدف اسکالر می سازیم $F = \lambda \times \text{Accuracy} + (1 - \lambda) \times \text{Diversity}$. سپس الگوریتم بهینه سازی HSA یا MA را به عنوان یک بهینه ساز تک هدفه روی F اعمال می کنیم. مزیت این روش سادگی آن است - کفایت الگوریتم را تقریباً بدون تغییر خاصی پیاده سازی کرد و فقط هنگام ارزیابی راه حل به جای بردار دو عددی، یک مقدار عددی برگرداند. همچنین می توان با تغییر وزن λ راه حل های مختلفی روی جبهه پارتو را پیدا کرد (با چندین اجرای مجزا). این روش در مهندسی بسیار رایج بوده به خاطر سهولت فهم و پیاده سازی. با این حال چند عیب دارد:

- **انتخاب وزن مناسب دشوار است:** تعیین λ تاثیر زیادی بر خروجی دارد و عملاً ترجیحات تصمیم گیرنده را نیاز دارد. اگر خیلی بالا باشد فقط دقت بهینه می شود؛ اگر خیلی پایین باشد فقط تنوع.

- **یافتن همه راه حل های پارتو با یک اجرا ممکن نیست:** هر اجرای وزن دار فقط یک راه حل بهینه (نقطه ای روی جبهه) می دهد. برای رسم جبهه کامل باید الگوریتم را برای وزن های گوناگون چندین بار اجرا کرد. این می تواند از نظر زمانی پرهزینه باشد. همچنین روش وزن دار در مسائل با جبهه غیر کوژ ممکن است برخی نواحی پارتو را هرگز پیدا نکند.

- با این وجود، برای مقایسه در تمرین، خواسته شده که روش ترکیب وزن دار نیز پیاده سازی و بررسی شود.

ما ۴ مدل تولید می‌کنیم:

• HSA-pareto

• HSA-weighted

• MA-pareto

• MA-weighted

و آن‌ها را با MF baseline مقایسه می‌کنیم.

تفاوت Pareto و Weighted (اصل trade-off)

• **Weighted-sum**: یک اسکالر می‌سازد که نتیجه‌اش معمولاً یک «نقطه مصالحه» است؛

یعنی مدل به یک ناحیه خاص از فضای (Accuracy, Diversity) گرایش پیدا می‌کند و به وزن‌ها حساس است.

• **Pareto** (غلبه پارتو): به جای یک جواب، یک جبهه از جواب‌های غیرمغلوب می‌سازد

(کاندیدهای مختلف برای trade-off های مختلف) و دقیقاً خروجی «جبهه پارتو نهایی (دقت در برابر تنوع)» را پوشش می‌دهد.

```
EA=MA,pareto: 100% ██████████ 916/916 [43:56<00:00, 2.58s/user]

=== Runtime ===
Setting: MA / pareto
Total runtime: 2636.62s | users: 916 | avg/user: 2.8775s
✓ Saved: /content/drive/MyDrive/Movielens_DataSet/results_ea/ea_MA_pareto_top10_from_top300_candidates.csv
EA=MA,weighted: 100% ██████████ 916/916 [33:33<00:00, 2.20s/user]

=== Runtime ===
Setting: MA / weighted
Total runtime: 2013.47s | users: 916 | avg/user: 2.1973s
✓ Saved: /content/drive/MyDrive/Movielens_DataSet/results_ea/ea_MA_weighted_top10_from_top300_candidates.csv
{('HSA',
  'pareto'): '/content/drive/MyDrive/Movielens_DataSet/results_ea/ea_HSA_pareto_top10_from_top300_candidates.csv',
 ('HSA',
  'weighted'): '/content/drive/MyDrive/Movielens_DataSet/results_ea/ea_HSA_weighted_top10_from_top300_candidates.csv',
 ('MA',
  'pareto'): '/content/drive/MyDrive/Movielens_DataSet/results_ea/ea_MA_pareto_top10_from_top300_candidates.csv',
 ('MA',
  'weighted'): '/content/drive/MyDrive/Movielens_DataSet/results_ea/ea_MA_weighted_top10_from_top300_candidates.csv'}
```

```

EA=HSA_pareto: 100% ██████████ 916/916 [05:35<00:00, 3.29user/s]

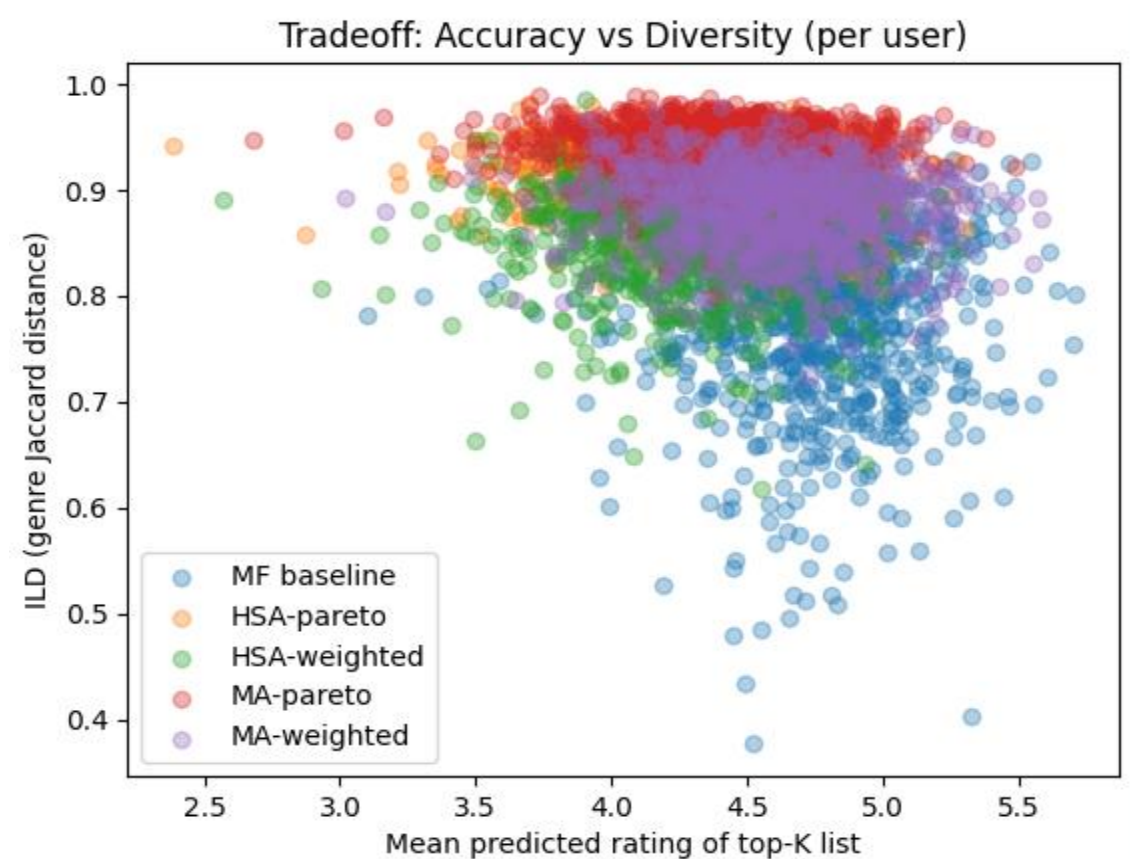
=== Runtime ===
Setting: HSA / pareto
Total runtime: 335.79s | users: 916 | avg/user: 0.3656s
✓ Saved: /content/drive/MyDrive/Movielens_DataSet/results_ea/ea_HSA_pareto_top10_from_top300_candidates.csv

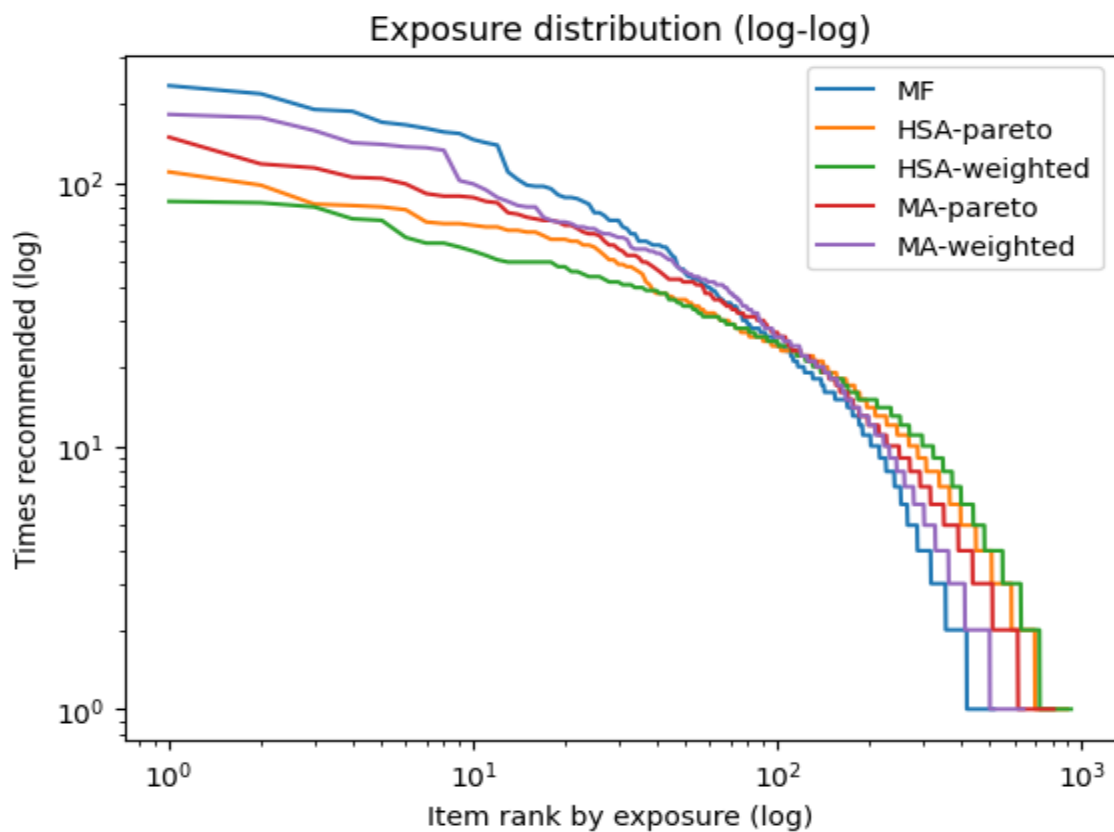
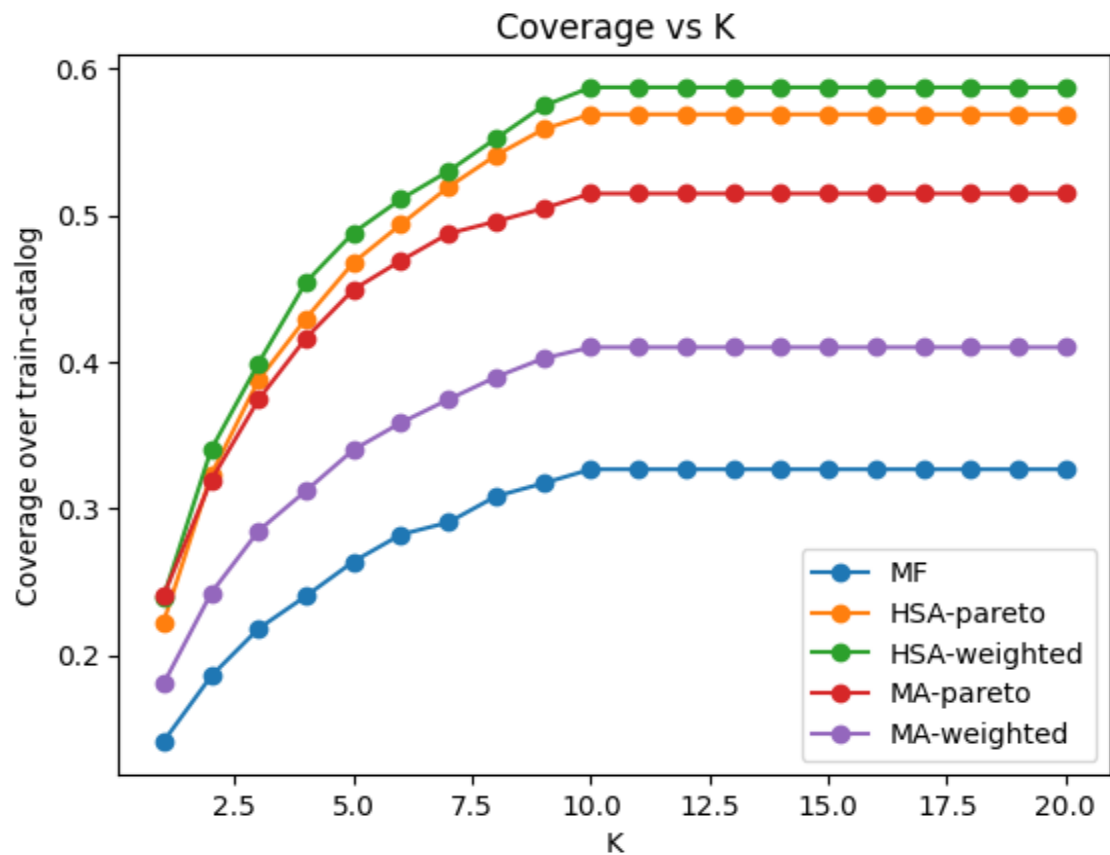
EA=HSA_weighted: 100% ██████████ 916/916 [38:59<00:00, 2.51s/user]

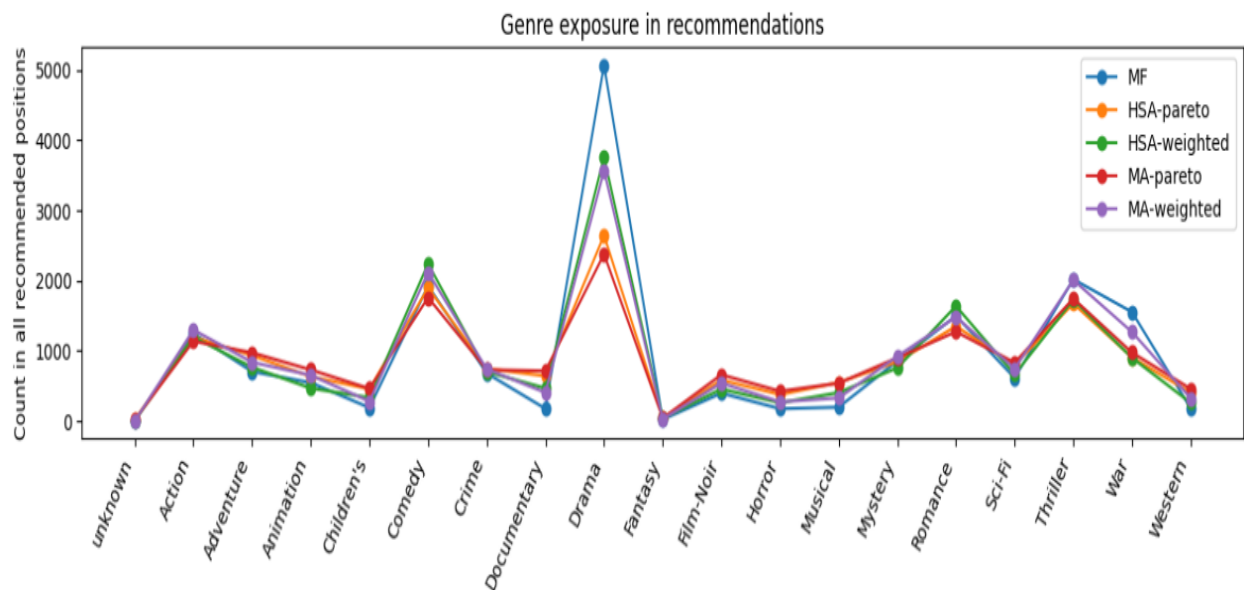
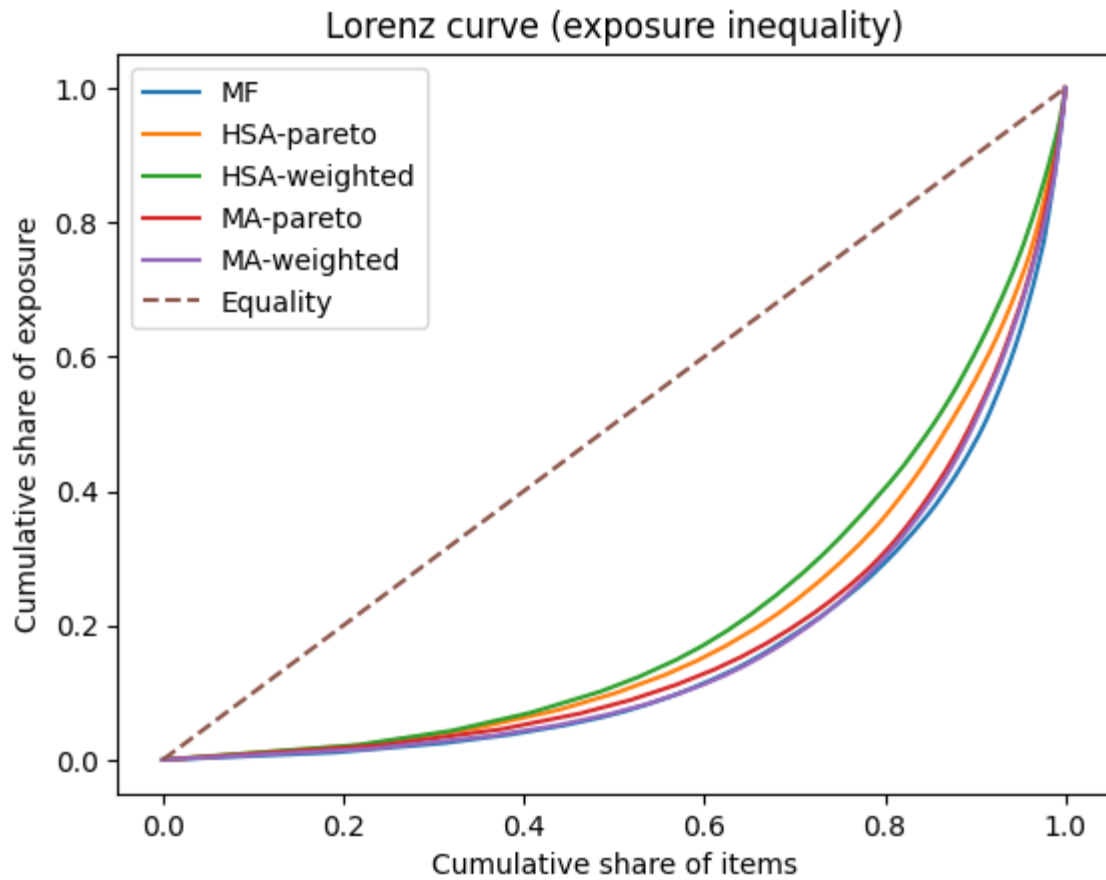
=== Runtime ===
Setting: HSA / weighted
Total runtime: 2339.66s | users: 916 | avg/user: 2.5533s
✓ Saved: /content/drive/MyDrive/Movielens_DataSet/results_ea/ea_HSA_weighted_top10_from_top300_candidates.csv

```

Model	Precision@10	Recall@10	NDCG@10	MRR@10	ILD	Coverage@10 (train)	Coverage@10 (all)	Gini	Entropy_norm	Users_evaluated	Unique_items_recommended
MA-pareto	0.017031	0.022122	0.020646	0.043345	0.940676	0.514940	0.481570	0.647300	0.884083	916	810
HSA-pareto	0.015175	0.017456	0.018076	0.042037	0.920432	0.568976	0.532105	0.604516	0.903679	916	895
MA-weighted	0.023362	0.031207	0.029602	0.066875	0.881359	0.410045	0.383472	0.663871	0.872651	916	645
HSA-weighted	0.016703	0.020233	0.021905	0.055706	0.863171	0.587413	0.549346	0.571875	0.917017	916	924
MF baseline	0.028493	0.037138	0.035428	0.077686	0.789337	0.326764	0.305589	0.674568	0.860417	916	514







Example user: 1					
	model	rank	item_id	title	genres
0	MF baseline	1	511	Lawrence of Arabia (1962)	Adventure, War
1	MF baseline	2	302	L.A. Confidential (1997)	Crime, Film-Noir, Mystery, Thriller
2	MF baseline	3	357	One Flew Over the Cuckoo's Nest (1975)	Drama
3	MF baseline	4	709	Strictly Ballroom (1992)	Comedy, Romance
4	MF baseline	5	318	Schindler's List (1993)	Drama, War
5	MF baseline	6	657	Manchurian Candidate, The (1962)	Film-Noir, Thriller
6	MF baseline	7	647	Ran (1985)	Drama, War
7	MF baseline	8	408	Close Shave, A (1995)	Animation, Comedy, Thriller
8	MF baseline	9	963	Some Folks Call It a Sling Blade (1993)	Drama, Thriller
9	MF baseline	10	474	Dr. Strangelove or: How I Learned to Stop Worr...	Sci-Fi, War
10	HSA-pareto	1	302	L.A. Confidential (1997)	Crime, Film-Noir, Mystery, Thriller
11	HSA-pareto	2	1084	Anne Frank Remembered (1995)	Documentary
12	HSA-pareto	3	963	Some Folks Call It a Sling Blade (1993)	Drama, Thriller
13	HSA-pareto	4	601	For Whom the Bell Tolls (1943)	Adventure, War
14	HSA-pareto	5	652	Rosencrantz and Guildenstern Are Dead (1990)	Comedy, Drama
15	HSA-pareto	6	708	Sex, Lies, and Videotape (1989)	Drama
16	HSA-pareto	7	380	Star Trek: Generations (1994)	Action, Adventure, Sci-Fi
17	HSA-pareto	8	435	Butch Cassidy and the Sundance Kid (1969)	Action, Comedy, Western
18	HSA-pareto	9	615	39 Steps, The (1935)	Thriller

: MF baseline

- Accuracy بالاتر (چون صرفاً Top-K بر اساس امتیاز MF است)
- ILD پایین تر (چون ممکن است ژانرهای تکراری/محبوب زیاد شوند)

EA ها:

- ILD بالاتر (به قیمت کمی افت Accuracy)
- همچنین تفاوت بین Pareto و Weighted :
 - Pareto : نقاط پخش شده تر و پوشش بهتر مرز trade-off

- **Weighted** : نقاط متراکمتر حول یک مصالحه (با توجه به وزن‌ها)
- مدل **ILD** خیلی بالاتر ولی **Precision/NDCG** افت شدید داشته باشد یعنی تنوع را “زیادی” خریده و کیفیت رنکینگ را قربانی کرده.
- **Constraint_ok_rate** پایین باشد یعنی **repair/penalty** کافی نبوده یا الگوریتم به راه‌حل‌های نقض‌کننده قید می‌چسبد.
- **Coverage/Gini/Entropy** بهتر شود یعنی توصیه‌ها کمتر روی چند آیتم محبوب قفل شده‌اند (پخش‌شدگی بهتر در کاتالوگ)

۴- وظیفه چهارم: مقایسه گروهی و بحث نهایی

MA اگر زمان طولانی‌تری در اختیار داشته باشیم، معمولاً می‌تواند جواب‌های “پخته‌تر” بدهد (نزدیک‌تر به مرز trade-off) اما در محیطی مثل کولب و وقتی برای هر کاربر ۳۰۰ کاندید داریم، LS می‌تواند خیلی گران شود و باعث کندی/قطع و وصل هم بشود.

الگوریتم جستجوی هارمونی (HSA)

در کد HSA با دو اهرم اصلی کار می‌کند:

• **HMCR** : با چه احتمال از حافظه (Harmony Memory) نمونه‌برداری شود

• **PAR** : با چه احتمال “تنظیم” انجام شود (شبیه perturbation)

اثر اپراتوری:

• **HMCR** بالا یعنی exploitation زیاد (چسبیدن به الگوهای خوب قبلی)

• **PAR** بالاتر یعنی exploration بیشتر (جهش/تنظیم بیشتر)

- چون ساختار تولید جواب در HSA پوزیشن به پوزیشن است، معمولاً سریع است و هزینه‌اش عمدتاً از محاسبه $O(K^2)$ می‌آید، نه از اپراتورهای سنگین.
- همگرایی HSA معمولاً نرم و پایدار است (کم‌نوسان‌تر)
- ولی ممکن است زودتر به یک ناحیه "قابل قبول" برسد و تنوع ساختاری خیلی پیچیده تولید نکند مگر اینکه PAR/repair خوب طراحی شده باشد.

الگوریتم ممتیک (MA)

MA در پیاده‌سازی ترکیبی از:

- **Selection** (tournament)
- **Crossover** (ترکیب)
- **Mutation**
- **Local Search (LS)** با احتمال LS_PROB و چند گام LS_STEPS

اثر اپراتوری:

- بخش GA (انتخاب/ترکیب/جهش) یعنی جستجوی سراسری
- Local Search یعنی exploitation سنگین روی هر فرد که به ازای هر کاندید، چند همسایه می‌سازید و دوباره ILD/acc را حساب می‌کند که هزینه محاسباتی به شدت بالا می‌رود.

تنوع (Diversity) و پوشش/عدالت مواجهه (Coverage/Gini/Entropy)

- Pareto معمولاً پوشش راه حل ها و تنوع نتایج بیشتری می دهد.
- در نمودارها (Coverage, Lorenz, Genre exposure) مدل های EA نسبت به MF :
 - Coverage بالاتر
 - Lorenz نزدیک تر به خط برابری (کاهش نابرابری مواجهه)
 - Genre exposure متعادل تر نشان دهد یعنی EA واقعاً از نظر تنوع سیستماتیک بهتر عمل کرده (و فقط ILD را بالا نبرده)

همگرایی (Convergence)

نمودار پیشرفت :

- در **Weighted** : همگرایی را می توان با بهترین/میانگین مقدار هدف اسکالر در نسل ها گزارش کرد (پایدارتر و ساده تر).
- در **Pareto** : همگرایی را با شاخص هایی مثل:
 - بهبود best Accuracy در جبهه
 - بهبود best Diversity در جبهه
 - تغییرات اندازه/کیفیت جبهه (مثلاً تعداد non-dominated ها)

کدام مناسب تر است؟

– اگر هدف «سیستم توصیه گر چندهدفه عملی و قابل اجرا در مقیاس» باشد:

- **HSA-pareto** معمولاً بهترین انتخاب “مهندسی” است:

- سرعت خیلی بهتر

- تولید trade-off واقعی (به جای چسبیدن به یک وزن خاص)

- مناسب برای تحلیل جبهه پارتو و ارائه گزارش

– اگر هدف «بهترین کیفیت ممکن» باشد و زمان اهمیتی نداشته باشد:

- **MA-pareto** از نظر تئوری می تواند بهترین مرز trade-off را بدهد،

اما در تنظیمات فعلی ($N=300$ ، کاربران زیاد، LS فعال) هزینه اش آن قدر بالا می شود که ممکن است عملی نباشد.

– **Weighted** در برابر **Pareto** :

- **Weighted** برای ارائه “یک نسخه نهایی” با سیاست ثابت (مثلاً محصول واقعی با وزن های مشخص) خوب است،

- اما برای اینجا که باید trade-off و جبهه پارتو را تحلیل کنیم، Pareto طبیعی تر و قابل دفاع تر است.

- [1] <https://github.com>
- [2] <https://stackoverflow.com/questions>
- [3] <https://colab.research.google.com/>
- [4] <https://www.researchgate.net/publication/348194344>
- [5] <https://chatgpt.com/>
- [6] <https://gemini.google.com/>